PLANNEX

A visual, step-by-step guide to the student planner code

Learn how the HTML builds the screens, how CSS styles them, and how JavaScript makes the planner work and remember changes.

This updated edition explains student-selected class times, editable grade weights, and the allowance-based 50 / 30 / 20 suggestion.

Files covered

- index.html — the app's structure, navigation, forms, tables, and page content
- styles.css — the color system, layout, components, and responsive behavior
- app.js — sample data, calculations, rendering, interactions, and browser storage
- CODE_GUIDE.md — a quick map for finding the part you want to edit

Use the examples to understand the ideas. Use the editable files in outputs/plannex when making changes.

Contents

1. How the three files work together 3

2. Finding the project structure 3

3. HTML: the page shell and brand 3

4. HTML: page sections and navigation 4

5. HTML: forms, buttons, and accessible labels 4

6. CSS: the color palette and design tokens 4

7. CSS: layout, cards, and responsive rules 4

8. JavaScript: the private wrapper and starter data 5

9. JavaScript: saved state and safe text 5

10. JavaScript: small helpers and date text 5

11. JavaScript: how rendering works 6

12. Daily planner: one list, several views 6

13. Class schedule: choosing an exact start time 6

14. Budget: allowance, spending, and 50 / 30 / 20 7

15. Grade calculator: row grades and category weights 7

16. Navigation and event listeners 8

17. Shared dialog: add, cancel, and save 8

18. Reset, first page, and troubleshooting 8

19. A safe, practical editing workflow 9

20. Full source listings 9

Appendix - A - HTML structure 10

Appendix - B - CSS visual system 16

Appendix - C - JavaScript behavior 47

1. How the three files work together

The browser reads the three source files as one app. A helpful mental model is: HTML names the parts, CSS arranges and decorates them, JavaScript supplies data and behavior.

Three connected code examples

```
<!-- index.html: a visible control with an id -->
<div id="budget-total">₱0</div>

/* styles.css: make that part look like a card */
.budget-stat { border-radius: 16px; padding: 20px; }

// app.js: find the part and put a value inside it
document.querySelector("#budget-total").textContent = "₱12,000";
```

An id connects a visible HTML element to JavaScript. A class such as budget-stat connects an element to CSS. If you rename an id, update every JavaScript query that uses it; otherwise the script will look for a part that no longer exists.

- Start at index.html when changing wording, labels, page sections, or controls.
- Start at styles.css when changing colors, spacing, sizing, or mobile layout.
- Start at app.js when changing stored data, calculations, actions, or what appears on screen.

2. Finding the project structure

The planner is a static browser app. It does not need a server database. Keep the three main files together so index.html can find styles.css and app.js.

File layout

```
outputs/plannex/
├── index.html
├── styles.css
├── app.js
├── CODE_GUIDE.md
├── Plannex-Code-Learning-Guide-Updated.pdf
└── assets/
    └── plannex-logo.png
```

The browser loads the stylesheet from the head of index.html and loads app.js near the end of the body. The logo path is relative to index.html, so the image file belongs in assets/.

3. HTML: the page shell and brand

The opening lines describe the document type, language, mobile viewport, page title, fonts, icon, and stylesheet. The sidebar then provides the brand logo and navigation.

Representative structure from index.html

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<link rel="stylesheet" href="styles.css" />
...
<a class="brand" href="#overview" aria-label="Plannex home">
  <img src="assets/plannex-logo.png" alt="Plannex logo" />
  <span>PLANN<span class="brand-accent">EX</span></span>
</a>
```

The viewport lets the layout respond to a phone's width. The image alt text describes the logo to people using a screen reader or when the image cannot load. The href points to the Overview hash.

Each navigation link has a data-page value such as data-page="budget". JavaScript reads that value and activates the section with id="page-budget".

4. HTML: page sections and navigation

Each screen is a section with page-section as its shared class. Its id follows the pattern page-name. Navigation changes which one has the active-page class.

A navigation link and the section it opens

```
<a class="nav-link" href="#classes" data-page="classes">
  Class schedule
</a>
...
<section class="page-section" id="page-classes">
  <h1>Class schedule.</h1>
  <button data-action="add-class">  Add a class</button>
  <div class="timetable" id="timetable"></div>
</section>
```

The timetable starts empty in HTML because JavaScript builds its rows from the saved class list. The same idea is used for tasks, expenses, grade rows, and the 50/30/20 amounts.

5. HTML: forms, buttons, and accessible labels

Buttons that open the shared dialog carry a data-action. Controls inside that dialog are generated by JavaScript. Labels, ids, button types, and accessible names help the controls remain understandable and usable.

Cancel has a different job from Save

```
<button type="button" data-close-dialog>Cancel</button>
<button type="submit">Save</button>

<label for="field-time">Start time</label>
<input id="field-time" name="time" type="time" required />
```

Cancel and the close button only close the dialog. They are explicitly type="button", so they cannot submit the form. Save is type="submit" and reaches the form's submit handler. The class start-time control uses the browser's time picker, allowing the student to choose an exact hour and minute.

6. CSS: the color palette and design tokens

At the top of styles.css, :root defines variables. A variable is a named value that can be reused throughout the stylesheet.

Palette tokens

```
:root {
  --midnight: #17182F;
  --surface: #242642;
  --green: #8DDA7A;
  --cloud: #F5F7FA;
  --slate: #667085;
}

body {
  background: var(--cloud);
  color: var(--midnight);
}
```

If the brand green changes, edit --green once. Components using var(--green) will receive the new color together. This is safer and easier than searching for repeated color codes.

7. CSS: layout, cards, and responsive rules

CSS classes describe appearance, not the app's saved information. Grid and flexbox arrange related items. Media queries adjust layout when the available screen becomes narrow.

Small-screen layout examples

```
@media (max-width: 700px) {
  .budget-rule-grid { grid-template-columns: 1fr; }
  .grade-weight-grid { grid-template-columns: repeat(2, minmax(0, 1fr)); }
  .table-scroll { overflow-x: auto; }
}
```

The budget suggestions stack in one column on small screens. Grade weight boxes use two columns. The timetable and assessment table can scroll sideways instead of squeezing their text into unreadable widths.

8. JavaScript: the private wrapper and starter data

The opening (() => { ... })() function is an immediately invoked function expression. It runs once and keeps helper names out of the browser's global scope. The first numbered section contains settings and sample records.

Shortened example of section 1

```
const KEY = "plannex-student-planner-v1";
const DAYS = ["Mon", "Tue", "Wed", "Thu", "Fri"];
const GRADE_CATEGORIES = ["Exam", "Performance Task", "Quiz", "Long Quiz", "Assignment"];
const seed = {
  allowance: 12000,
  gradeWeights: { Exam: 40, "Performance Task": 40, Quiz: 10, "Long Quiz": 0, Assignment: 10 },
  tasks: [],
  classes: [],
  expenses: [],
  grades: [],
};
```

The real seed object includes realistic sample tasks, weekly classes, expenses, and assessment scores. Change it to edit fresh-install examples. Existing saved data is not replaced just because seed changes; the reset action intentionally restores the sample data.

9. JavaScript: saved state and safe text

The state object is the app's current working copy of the planner. On startup the app tries to read saved JSON. If none exists or the data cannot be parsed, it uses the starter data.

The storage and escaping pattern

```
const saved = JSON.parse(localStorage.getItem(KEY));
state = saved ? { ...structuredClone(seed), ...saved } : structuredClone(seed);
const persist = () => localStorage.setItem(KEY, JSON.stringify(state));

const escapeHtml = (value) => String(value).replace(/[&<>"']/g, ...);
```

localStorage saves text for this browser and origin. JSON.stringify turns the state into text; JSON.parse reads it again. structuredClone gives the app a separate copy of the sample data. escapeHtml protects user-entered class names and expense descriptions when they are inserted into HTML.

> The app keeps data on the current device/browser. It does not synchronize entries to another device or account.

10. JavaScript: small helpers and date text

Small helpers prevent the same work from being copied throughout the code.

Short helper examples

```
const $ = (selector, root = document) => root.querySelector(selector);
const $$ = (selector, root = document) => [...root.querySelectorAll(selector)];
const makeId = () => `${Date.now().toString(36)}${Math.random().toString(36).slice(2, 7)}`;

function formatPercent(value) {
  return `${Number(Number(value).toFixed(1))}%`;
}
```

$ returns the first matching element. $$ returns all matches in an array. makeId gives a new task or class an identifier. formatPercent rounds its display to one decimal place without changing the stored percentage.

11. JavaScript: how rendering works

Render functions take state and update the page. They do not ask the page what the saved planner should be; the state object is the source of truth.

The main render pass

```
function render() {
  renderTasks();
  renderBudget();
  renderGrades();
  renderClasses();
  renderWeek();
  persist();
}
```

After an edit, render refreshes the screens from state and saves the snapshot. This keeps the Overview totals, the full feature page, and localStorage in sync. Individual render functions handle tasks, budget, grades, classes, and the compact week strip.

12. Daily planner: one list, several views

tasksMarkup creates the repeating HTML for one task row. renderTasks counts completed and remaining tasks, updates progress, and applies the selected All, To do, or Completed filter.

Filter and count logic

```
const remaining = state.tasks.filter((task) => !task.done).length;
const done = state.tasks.length - remaining;
const filter = $(".filter-tab.selected")?.dataset.filter || "all";
const visible = state.tasks.filter((task) =>
  filter === "all" || (filter === "open" ? !task.done : task.done)
);
```

The optional chaining operator ?. means 'only read dataset.filter if a selected tab exists.' When a student marks a task complete or removes it, the action changes state and then calls render.

13. Class schedule: choosing an exact start time

The Add a class dialog includes a browser time input. The browser returns a 24-hour value such as 13:30. The app normalizes saved values so sorting does not depend on whether an older value used AM/PM text.

Input and saved class example

```
field("Start time", "time", "time", "09:00")

state.classes.push({
  id: makeId(),
  day: data.get("day"),
  time: normalizeClassTime(data.get("time")),
  name: data.get("name").trim(),
  room: data.get("room").trim() || "Room not set",
});
```

normalizeClassTime converts a selected time to HH:mm. The timetable combines the standard rows with the actual times in the student's class list, removes duplicate times, and sorts them. That means a 9:35 class gets its own row between 9:00 and 10:00. Classes that share a time stay in the same day cell. The class repeats each week on its chosen weekday.

> For comparison, toMinutes turns 13:30 into 810 minutes after midnight. Comparing 810 with other minute counts is a simple way to sort times and select today's next class.

14. Budget: allowance, spending, and 50 / 30 / 20

renderBudget adds recorded expense amounts, subtracts them from the monthly allowance, then displays a suggested split of the allowance. The suggestions update when the student edits the allowance.

Allowance split calculation

```
const allowance = Number(state.allowance) || 0;
const spent = state.expenses.reduce((sum, expense) => sum + Number(expense.amount), 0);
const remaining = allowance - spent;

$("#rule-needs").textContent = money(allowance * 0.5);
$("#rule-wants").textContent = money(allowance * 0.3);
$("#rule-savings").textContent = money(allowance * 0.2);
```

The rule is a flexible suggestion. For ₱12,000 each month: needs receive ₱6,000, wants ₱3,600, and savings ₱2,400. This sums to ₱12,000. The 50/30/20 percentages apply to the allowance, not to expenses already entered.

If the student changes the allowance to ₱8,000, the suggested amounts recalculate to ₱4,000, ₱2,400, and ₱1,600. Expense entries remain their own records.

15. Grade calculator: row grades and category weights

Each assessment row has a category, score earned, and possible points. The item's raw percentage is score divided by possible points, times 100. Multiple scored items in the same category are combined by total points earned over total points possible.

One item: score divided by possible points

```
function getRowGrade(row) {
  const score = Number(row.score);
  const total = Number(row.total);
  return Number.isFinite(score) && Number.isFinite(total) && total > 0
    ? (score / total) * 100
    : null;
}
```

Example: 18 out of 20 is 90%. If a category has two quizzes, 18/20 and 15/20, the category average is (18+15)/(20+20) = 82.5%. This lets a 100-point test count more than a 10-point quiz within that same category.

Students enter the course-category percentages from their syllabus. The starting examples are Exam 40%, Performance Task 40%, Quiz 10%, Long Quiz 0%, Assignment 10%. All five values must add to 100% before Calculate works.

Weighted result formula (plain-language version)

```
weighted points = category percentage × category weight
current weighted grade = sum(weighted points) ÷ sum(weights with scores)
```

If Exams count 40% and the exam average is 80%, the exam contribution is 80 × 0.40 = 32 course points. If Performance Tasks count 40% and average 90%, they contribute 90 × 0.40 = 36 course points. With only those two categories scored, the current grade is (32 + 36) ÷ 0.80 = 85%. The app also reports that scored work covers 80% of the course weight. Once all weighted categories have scores, the denominator is 100% and the result reflects the full course.

A category with a 0% weight does not change the result. Changing a weight or score clears the old calculated display so the student can calculate the updated values again.

16. Navigation and event listeners

The navigation handler reads data-page, toggles active-page and active classes, updates the heading, and changes the URL hash. The browser's back/forward address can therefore identify the current section.

Direct and delegated click handlers

```
link.addEventListener("click", (event) => {
  event.preventDefault();
  navigate(link.dataset.page);
});

document.addEventListener("click", (event) => {
  const target = event.target.closest("[data-action]");
  if (target) openDialog(target.dataset.action);
});
```

Some controls are rebuilt after every render. A delegated listener on document can still catch their clicks because the event travels upward from the clicked button. This avoids attaching duplicate listeners each time the table or task list is redrawn.

17. Shared dialog: add, cancel, and save

field creates a labeled input or dropdown. openDialog chooses a configuration based on the action. Each configuration has a title, fields, and a save function. The form submit handler validates required fields, calls save, redraws the app, closes the dialog, and displays a toast.

Save and Cancel have separate paths

```
form.onsubmit = (event) => {
  event.preventDefault();
  if (!form.reportValidity()) return;
  const data = new FormData(form);
  config.save(data);
  render();
  dialog.close();
};

document.addEventListener("click", (event) => {
  if (event.target.closest("[data-close-dialog]")) dialog.close();
});
```

Cancel never reaches form.onsubmit. It closes the dialog and leaves state unchanged. This explicit separation also prevents an accidental task from being saved when someone chooses to opt out.

18. Reset, first page, and troubleshooting

At startup, the code reads the URL hash, opens an allowed page, and performs the first render. The Reset sample data button asks for confirmation before replacing the current state.

Startup sequence

```
const initial = location.hash.replace("#", "");
navigate(["overview", "planner", "classes", "budget", "grades"].includes(initial)
  ? initial
  : "overview");
render();
```

If a change does not appear, check the browser console for an error, then verify that the element id in HTML exactly matches the selector in JavaScript. If you rename a grade category, update GRADE_CATEGORIES and seed.gradeWeights, then check any existing rows and weights.

- A broken image usually means its src path does not match the actual logo file path.
- A page that does not open may have a mismatched data-page, page id, or allowed-page name.

- A blank grade result may mean the weights do not total 100%, no scored category has positive weight, or the student has not pressed Compute grade after editing.
- A class at a surprising hour may be a legacy saved time that needs migration; new input uses the browser time picker.

19. A safe, practical editing workflow

Make one small change at a time. Save the file, reload the planner, and check both a normal case and a cancel/empty case where relevant.

- Before editing, make a copy of the file you plan to change.
- For text, search in index.html. For visual styling, search the related CSS comment or class. For behavior, search for the JavaScript function or data-action.
- Keep existing ids and data attributes unless you are updating their JavaScript/CSS references too.
- Check a narrow window after layout changes. Check saved state after changing data structures.
- Use CODE_GUIDE.md as the short edit map; use this PDF to learn the surrounding logic.

20. Full source listings

The following appendices reproduce the current editable HTML, CSS, and JavaScript from outputs/plannex. Line numbers and visual wraps are for navigation. For edits, use the source files so long lines and indentation remain intact.

> The full listings are included so you can connect each lesson to the actual project, not just to shortened examples.

Appendix - A - HTML structure

Source file: index.html. The listing below is read from the project file when this guide is generated. It has 315 source lines.

Visual line wraps may split a long source line across several printed rows. The numbered source line remains one line in the editable file.

index.html - lines 1-52

```
0001  <!doctype html>
0002  <html lang="en">
0003    <head>
0004      <meta charset="utf-8">
0005      <meta name="viewport" content="width=device-width, initial-scale=1">
0006      <meta name="theme-color" content="#17182F">
0007      <meta name="description" content="Plannex is your calm, everyday student planner for classes,
      tasks, spending, and grades.">
0008      <title>Plannex — your student planner</title>
0009
0010      <!-- Fonts and the main visual style sheet. -->
0011      <link rel="preconnect" href="https://fonts.googleapis.com">
0012      <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
0013      <link href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Manrope
      :wght@400;500;600;700;800&display=swap" rel="stylesheet">
0014      <link rel="stylesheet" href="styles.css">
0015    </head>
0016    <body>
0017      <div class="app-shell">
0018        <!-- Shared sidebar: the data-page names must match the page section IDs below. -->
0019        <aside class="sidebar" id="sidebar">
0020          <a class="brand" href="#overview" aria-label="Plannex home">
0021            <span class="brand-mark"><svg viewBox="300 110 420 420" aria-hidden="true"><image
      href="assets/plannex-logo.png" width="1110" height="835"></image></svg></span><span
      class="brand-name">PLANN<span class="brand-green">E</span>X<small>Make every day
      Count</small></span>
0022          </a>
0023
0024          <p class="nav-label">Workspace</p>
0025          <nav class="primary-nav" aria-label="Main navigation">
0026            <a class="nav-link active" href="#overview" data-page="overview"><span
      class="nav-icon"> </span>Overview</a>
0027            <a class="nav-link" href="#planner" data-page="planner"><span
      class="nav-icon"> </span>Daily planner</a>
0028            <a class="nav-link" href="#classes" data-page="classes"><span
      class="nav-icon"> </span>Class schedule</a>
0029            <a class="nav-link" href="#budget" data-page="budget"><span class="nav-icon"> </span>My
      budget</a>
0030            <a class="nav-link" href="#grades" data-page="grades"><span class="nav-icon"> </span>Grade
       calculator</a>
0031          </nav>
0032
0033          <div class="sidebar-bottom">
0034            <div class="sidebar-note">
0035              <span class="note-spark"> </span>
0036              <p>One thing at a time.<br><strong>You’ve got this.</strong></p>
0037              <span class="note-doodle"> </span>
0038            </div>
0039            <div class="profile">
0040              <div class="avatar">JS</div>
0041              <div><strong>Jamie Santos</strong><small>Student workspace</small></div>
0042              <span class="profile-dots">···</span>
0043            </div>
0044          </div>
0045        </aside>
0046
0047        <main class="main-area">
0048          <!-- Shared header: JavaScript updates the page name and today's date. -->
0049          <header class="topbar">
0050            <button class="mobile-menu" id="menu-toggle" aria-label="Open menu"> </button>
0051            <div class="breadcrumb">
0052              <span>My workspace</span>
```

index.html - lines 53-106

```
0053              <b>/</b>
0054              <strong id="crumb-current">Overview</strong>
0055            </div>
0056            <div class="topbar-right">
0057              <span class="today-chip"><span class="live-dot"></span><span
    id="today-label"></span></span>
0058              <button class="icon-button" id="reset-data" title="Restore sample data"
    aria-label="Restore sample data"> </button>
0059              <div class="avatar avatar-small">JS</div>
0060            </div>
0061          </header>
0062
0063          <div class="content-wrap">
0064            <!-- Overview: a quick summary of tasks, classes, budget, and study momentum. -->
0065            <section class="page-section active-page" id="page-overview">
0066              <div class="welcome-row">
0067                <div>
0068                  <p class="eyebrow" id="greeting">YOUR DAY, AT A GLANCE</p>
0069                  <h1>A little more <span>organized.</span></h1>
0070                  <p class="page-subtitle">A calm place to keep your student life together.</p>
0071                </div>
0072                <button class="button button-dark" data-action="add-task"><span> </span> Add a
    task</button>
0073              </div>
0074
0075              <div class="stats-grid">
0076                <article class="stat-card">
0077                  <div class="stat-top"><span>Tasks for today</span><span class="stat-icon
    icon-green"> </span></div>
0078                  <div class="stat-number" id="stat-tasks">0 <small>tasks left</small></div>
0079                  <div class="progress-track"><span id="task-progress"></span></div>
0080                  <div class="stat-foot" id="task-foot">Keep your momentum going</div>
0081                </article>
0082                <article class="stat-card">
0083                  <div class="stat-top"><span>Budget remaining</span><span class="stat-icon
    icon-peach"> </span></div>
0084                  <div class="stat-number" id="stat-budget">₱0</div>
0085                  <div class="stat-foot"><span class="mini-dot"></span><span id="budget-foot">of your
    monthly allowance</span></div>
0086                </article>
0087                <article class="stat-card">
0088                  <div class="stat-top"><span>Current raw grade</span><span class="stat-icon
    icon-lilac"> </span></div>
0089                  <div class="stat-number" id="stat-grade">— <small>%</small></div>
0090                  <div class="stat-foot">From your entered score totals</div>
0091                </article>
0092              </div>
0093
0094              <div class="dashboard-grid">
0095                <article class="panel today-panel">
0096                  <div class="panel-heading">
0097                    <div><p class="eyebrow">MAKE IT COUNT</p><h2>Today’s plan <span
    class="heading-count" id="today-count">0</span></h2></div>
0098                    <a href="#planner" class="text-link">See all <span>→</span></a>
0099                  </div>
0100                  <div class="task-list" id="overview-tasks"></div>
0101                  <button class="add-inline" data-action="add-task">  Add a task</button>
0102                </article>
0103
0104                <article class="panel schedule-panel">
0105                  <div class="panel-heading">
0106                    <div><p class="eyebrow">UP NEXT</p><h2>Your schedule</h2></div>
```

index.html - lines 107-163

```
0107                    <a href="#classes" class="text-link">Full week <span>→</span></a>
0108                  </div>
0109                  <div class="next-class" id="next-class"></div>
0110                </article>
0111
0112                <article class="panel spending-panel">
0113                  <div class="panel-heading">
0114                    <div><p class="eyebrow">MONEY CHECK-IN</p><h2>This month</h2></div>
0115                    <a href="#budget" class="text-link">Budget <span>→</span></a>
0116                  </div>
0117                  <div class="spend-total"><strong id="overview-spent">₱0</strong><span>spent so
      far</span></div>
0118                  <div class="budget-bar"><span id="budget-bar"></span></div>
0119                  <div class="budget-labels"><span id="spent-label">₱0 spent</span><span
      id="limit-label">₱0 limit</span></div>
0120                  <div class="recent-transactions" id="recent-transactions"></div>
0121                </article>
0122
0123                <article class="panel focus-panel">
0124                  <div class="focus-copy">
0125                    <span class="focus-badge">A SMALL REMINDER</span>
0126                    <h2>Progress is still progress, <em>even on the busy days.</em></h2>
0127                    <p>Pick one thing to start. The rest can follow.</p>
0128                  </div>
0129                  <div class="focus-art"><span class="art-sun"></span><span
      class="art-star"> </span><span class="art-plant"> </span></div>
0130                </article>
0131              </div>
0132            </section>
0133
0134            <!-- Daily planner: filter tasks by all, still to do, or completed. -->
0135            <section class="page-section" id="page-planner">
0136              <div class="welcome-row">
0137                <div>
0138                  <p class="eyebrow">YOUR EVERYDAY RESET</p>
0139                  <h1>Daily <span>planner.</span></h1>
0140                  <p class="page-subtitle">A simple list is a good place to begin.</p>
0141                </div>
0142                <button class="button button-dark" data-action="add-task">  Add a task</button>
0143              </div>
0144
0145              <div class="planner-layout">
0146                <article class="panel planner-main">
0147                  <div class="panel-heading">
0148                    <div><p class="eyebrow" id="planner-date">TODAY</p><h2>My to-do list</h2></div>
0149                    <span class="heading-count" id="planner-count">0</span>
0150                  </div>
0151                  <div class="filter-tabs">
0152                    <button class="filter-tab selected" data-filter="all">All</button>
0153                    <button class="filter-tab" data-filter="open">To do</button>
0154                    <button class="filter-tab" data-filter="done">Completed</button>
0155                  </div>
0156                  <div class="task-list full-task-list" id="planner-tasks"></div>
0157                </article>
0158
0159                <aside class="panel planner-side">
0160                  <p class="eyebrow">A GOOD RHYTHM</p>
0161                  <h2>Make space for breaks.</h2>
0162                  <p class="page-subtitle">Try a short focus session, then step away for a few
      minutes. Small resets add up.</p>
0163                  <div class="break-card"><span> </span><div><strong>Focus for 25
      minutes</strong><small>One task. No rush.</small></div></div>
```

index.html - lines 164-217

```
0164                <div class="week-mini" id="week-mini"></div>
0165              </aside>
0166            </div>
0167          </section>
0168
0169          <!-- Class schedule: JavaScript builds the weekly grid from the class data. -->
0170          <section class="page-section" id="page-classes">
0171            <div class="welcome-row">
0172              <div>
0173                <p class="eyebrow">YOUR WEEK, IN ONE PLACE</p>
0174                <h1>Class <span>schedule.</span></h1>
0175                <p class="page-subtitle">Know where you need to be, and when.</p>
0176              </div>
0177              <button class="button button-dark" data-action="add-class">  Add a class</button>
0178            </div>
0179
0180            <article class="panel timetable-panel">
0181              <div class="panel-heading">
0182                <div><p class="eyebrow">SEMESTER 1 · 2026</p><h2>Weekly timetable</h2></div>
0183                <div class="schedule-legend"><span><i class="legend-dot
    dot-green"></i>Classes</span></div>
0184              </div>
0185              <div class="table-scroll"><div class="timetable" id="timetable"></div></div>
0186            </article>
0187            <p class="section-hint">Tip: classes repeat each week. Add your subjects and choose the
    exact start time from your schedule.</p>
0188          </section>
0189
0190          <!-- Budget: allowance and expenses are stored together in the planner state. -->
0191          <section class="page-section" id="page-budget">
0192            <div class="welcome-row">
0193              <div>
0194                <p class="eyebrow">SPEND WITH A LITTLE MORE CLARITY</p>
0195                <h1>My <span>budget.</span></h1>
0196                <p class="page-subtitle">Keep track of what comes in and where it goes.</p>
0197              </div>
0198              <button class="button button-dark" data-action="add-expense">  Add expense</button>
0199            </div>
0200
0201            <div class="budget-stats">
0202              <article class="stat-card budget-stat">
0203                <span>Monthly allowance</span>
0204                <strong id="budget-total">₱0</strong>
0205                <button class="subtle-button" data-action="edit-budget">Edit allowance →</button>
0206              </article>
0207              <article class="stat-card budget-stat"><span>Total spent</span><strong
    id="budget-spent">₱0</strong><small>This month</small></article>
0208              <article class="stat-card budget-stat"><span>Left to spend</span><strong
    id="budget-remaining">₱0</strong><small id="budget-remaining-note">You’re doing
    great</small></article>
0209            </div>
0210
0211            <article class="panel budget-rule-panel">
0212              <div class="panel-heading">
0213                <div><p class="eyebrow">A FLEXIBLE STARTING POINT</p><h2>Suggested 50 / 30 / 20
    split</h2></div>
0214                <span class="rule-total">From <strong id="rule-allowance">₱0</strong></span>
0215              </div>
0216              <p class="rule-intro">A simple guide for dividing your monthly allowance. Adjust it to
     fit your school costs and needs.</p>
0217              <div class="budget-rule-grid">
```

index.html - lines 218-270

```
0218                 <article class="rule-card rule-needs"><span>50% · NEEDS</span><strong
      id="rule-needs">₱0</strong><small>Food, transport, school essentials</small></article>
0219                 <article class="rule-card rule-wants"><span>30% · WANTS</span><strong
      id="rule-wants">₱0</strong><small>Fun, treats, and personal extras</small></article>
0220                 <article class="rule-card rule-savings"><span>20% · SAVINGS</span><strong
      id="rule-savings">₱0</strong><small>Save for goals or unexpected costs</small></article>
0221               </div>
0222               <p class="rule-note">These amounts are suggestions based on your allowance, not
      spending limits.</p>
0223             </article>
0224
0225             <article class="panel expenses-panel">
0226               <div class="panel-heading">
0227                 <div><p class="eyebrow">THE LITTLE THINGS ADD UP</p><h2>Recent expenses</h2></div>
0228                 <select id="expense-filter" class="select-small" aria-label="Filter expenses">
0229                   <option value="all">All categories</option>
0230                   <option>Food</option>
0231                   <option>Transport</option>
0232                   <option>School</option>
0233                   <option>Personal</option>
0234                   <option>Other</option>
0235                 </select>
0236               </div>
0237               <div id="expense-list" class="expense-list"></div>
0238             </article>
0239           </section>
0240
0241           <!-- Grade calculator: set category weights, enter scores, then compute a weighted grade.
      -->
0242           <section class="page-section" id="page-grades">
0243             <div class="welcome-row">
0244               <div>
0245                 <p class="eyebrow">KEEP AN EYE ON YOUR PROGRESS</p>
0246                 <h1>Grade <span>calculator.</span></h1>
0247                 <p class="page-subtitle">Match the category percentages to your course syllabus,
      then enter your scores.</p>
0248               </div>
0249             </div>
0250
0251             <div class="grade-summary">
0252               <div>
0253                 <span class="eyebrow">CURRENT WEIGHTED GRADE</span>
0254                 <div class="grade-big" id="grade-average">—<small>%</small></div>
0255                 <p class="grade-caption" id="grade-caption">Enter scores and totals, then compute
      your grade.</p>
0256               </div>
0257               <div class="grade-summary-art"><span> </span></div>
0258             </div>
0259
0260             <article class="panel grade-weights-panel">
0261               <div class="panel-heading">
0262                 <div><p class="eyebrow">SET THIS FROM YOUR SYLLABUS</p><h2>Category
      weights</h2></div>
0263                 <span class="grade-weight-total" id="grade-weight-total">100%</span>
0264               </div>
0265               <p class="grade-weight-help">Enter how much each category counts toward your course
      grade. The weights must add up to 100% before calculation.</p>
0266               <div class="grade-weight-grid" id="grade-weights"></div>
0267               <p class="grade-weight-status" id="grade-weight-status" aria-live="polite"></p>
0268             </article>
0269
0270             <article class="panel grades-panel">
```

index.html - lines 271-315

```
0271               <div class="panel-heading">
0272                 <div><p class="eyebrow">ASSESSMENT ITEMS</p><h2>Grade tracker</h2></div>
0273                 <span class="grade-count" id="grade-count">0 rows</span>
0274               </div>
0275               <div class="grades-table-wrap">
0276                 <table class="grades-table">
0277                   <thead>
0278                     <tr><th>CATEGORY</th><th>SCORE</th><th>POSSIBLE</th><th>ITEM
      GRADE</th><th></th></tr>
0279                   </thead>
0280                   <tbody id="grade-rows"></tbody>
0281                 </table>
0282               </div>
0283               <div class="grade-actions">
0284                 <button class="button button-quiet" id="add-grade-row" type="button">  Add
      row</button>
0285                 <button class="button button-dark" id="compute-grade" type="button">Compute
      grade</button>
0286               </div>
0287               <p class="grade-note">Each category grade uses its earned points ÷ possible points.
      The category result is multiplied by its syllabus weight.</p>
0288             </article>
0289           </section>
0290
0291           <footer class="footer-note">Made for the little things that make student life feel easier.
       <span> </span></footer>
0292         </div>
0293       </main>
0294     </div>
0295
0296     <!-- Shared feedback message and entry dialog used by task, class, and budget actions. -->
0297     <div class="toast" id="toast" role="status" aria-live="polite"></div>
0298     <dialog id="entry-dialog">
0299       <form id="entry-form">
0300         <div class="dialog-head">
0301           <div><p class="eyebrow" id="dialog-eyebrow">QUICK ADD</p><h2 id="dialog-title">Add a
      task</h2></div>
0302           <button class="close-dialog" type="button" data-close-dialog aria-label="Close">×</button>
0303         </div>
0304         <div id="dialog-fields"></div>
0305         <div class="dialog-actions">
0306           <button class="button button-quiet" type="button" data-close-dialog>Cancel</button>
0307           <button class="button button-dark" id="dialog-save" type="submit">Save</button>
0308         </div>
0309       </form>
0310     </dialog>
0311
0312     <!-- Load behavior after all page controls are in place. -->
0313     <script src="app.js"></script>
0314   </body>
0315 </html>
```

Appendix - B - CSS visual system

Source file: styles.css. The listing below is read from the project file when this guide is generated. It has 1917 source lines.

Visual line wraps may split a long source line across several printed rows. The numbered source line remains one line in the editable file.

styles.css - lines 1-62

```
0001  /* Brand palette and shared design tokens */
0002
0003  :root {
0004    --midnight: #17182F;
0005    --surface: #242642;
0006    --green: #8DDA7A;
0007    --cloud: #F5F7FA;
0008    --slate: #667085;
0009    --ink: #24263d;
0010    --muted: #898da0;
0011    --line: #eceef3;
0012    --white: #fff;
0013    --peach: #f8e3d1;
0014    --lilac: #e8e1fa;
0015    --mint: #e5f5df;
0016    --shadow: 0 8px 28px rgba(23,24,47,.045);
0017  }
0018
0019  * {
0020    box-sizing: border-box;
0021  }
0022
0023  html {
0024    scroll-behavior: smooth;
0025  }
0026
0027  body {
0028    margin: 0;
0029    background: var(--cloud);
0030    color: var(--ink);
0031    font-family: 'DM Sans',sans-serif;
0032    font-size: 14px;
0033    -webkit-font-smoothing: antialiased;
0034  }
0035
0036  button,input,select {
0037    font: inherit;
0038  }
0039
0040  button,a {
0041    -webkit-tap-highlight-color: transparent;
0042  }
0043
0044  button {
0045    cursor: pointer;
0046  }
0047
0048  .app-shell {
0049    min-height: 100vh;
0050  }
0051
0052  .sidebar {
0053    width: 244px;
0054    position: fixed;
0055    inset: 0 auto 0 0;
0056    background: var(--midnight);
0057    color: #fff;
0058    padding: 28px 18px 18px;
0059    display: flex;
0060    flex-direction: column;
0061    z-index: 20;
0062  }
```

styles.css - lines 63-124

```
0063
0064  /* Brand lockup and fixed navigation shell */
0065
0066  .brand {
0067    display: flex;
0068    align-items: center;
0069    gap: 11px;
0070    color: #fff;
0071    text-decoration: none;
0072    font: 800 22px Manrope,sans-serif;
0073    letter-spacing: -1px;
0074    padding: 0 12px 38px;
0075  }
0076
0077  .brand-mark {
0078    width: 40px;
0079    height: 40px;
0080    flex: none;
0081    overflow: hidden;
0082    border: 1px solid #343550;
0083    border-radius: 12px;
0084    background: var(--surface);
0085    box-shadow: 0 3px 11px #090a1b55;
0086  }
0087
0088  .brand-mark svg {
0089    display: block;
0090    width: 100%;
0091    height: 100%;
0092  }
0093
0094  .brand-name {
0095    font: 800 15px/1.1 Manrope,sans-serif;
0096    letter-spacing: 1.4px;
0097    color: #fff;
0098  }
0099
0100  .brand-green {
0101    color: var(--green);
0102  }
0103
0104  .brand-name small {
0105    display: block;
0106    margin-top: 5px;
0107    font: 400 9px/1 'DM Sans',sans-serif;
0108    letter-spacing: .15px;
0109    color: #a4a7b7;
0110  }
0111
0112  .nav-label,.eyebrow {
0113    font-size: 10px;
0114    letter-spacing: 1.5px;
0115    font-weight: 700;
0116    color: #a4a7b7;
0117    margin: 0;
0118  }
0119
0120  .nav-label {
0121    padding: 0 14px 12px;
0122    color: #797d94;
0123  }
0124
```

styles.css - lines 125-186

```
0125  .primary-nav {
0126    display: grid;
0127    gap: 5px;
0128  }
0129
0130  .nav-link {
0131    height: 43px;
0132    display: flex;
0133    align-items: center;
0134    gap: 13px;
0135    border-radius: 9px;
0136    color: #b9bbca;
0137    text-decoration: none;
0138    padding: 0 13px;
0139    font-size: 13px;
0140    font-weight: 500;
0141    transition: .2s;
0142  }
0143
0144  .nav-link:hover {
0145    background: #22233d;
0146    color: #fff;
0147  }
0148
0149  .nav-link.active {
0150    background: #2c2d48;
0151    color: #fff;
0152    font-weight: 600;
0153  }
0154
0155  .nav-icon {
0156    width: 17px;
0157    text-align: center;
0158    font-size: 17px;
0159    color: #a2a4b7;
0160  }
0161
0162  .nav-link.active .nav-icon {
0163    color: var(--green);
0164  }
0165
0166  .sidebar-bottom {
0167    margin-top: auto;
0168  }
0169
0170  .sidebar-note {
0171    position: relative;
0172    margin: 0 2px 20px;
0173    padding: 17px 15px;
0174    background: #22233d;
0175    border: 1px solid #30314c;
0176    border-radius: 12px;
0177    overflow: hidden;
0178  }
0179
0180  .sidebar-note p {
0181    font-size: 12px;
0182    line-height: 1.8;
0183    color: #bcbfce;
0184    margin: 0 0 0 18px;
0185  }
0186
```

styles.css - lines 187-248

```
0187  .sidebar-note strong {
0188    font-size: 13px;
0189    color: #fff;
0190  }
0191
0192  .note-spark {
0193    position: absolute;
0194    left: 13px;
0195    top: 17px;
0196    color: var(--green);
0197  }
0198
0199  .note-doodle {
0200    position: absolute;
0201    right: 18px;
0202    bottom: 14px;
0203    font-size: 28px;
0204    color: #666a86;
0205  }
0206
0207  .profile {
0208    display: flex;
0209    align-items: center;
0210    gap: 10px;
0211    padding: 15px 7px 0;
0212    border-top: 1px solid #2b2d48;
0213  }
0214
0215  .avatar {
0216    width: 33px;
0217    height: 33px;
0218    flex: none;
0219    border-radius: 50%;
0220    background: #e9e0d6;
0221    color: #655143;
0222    display: grid;
0223    place-items: center;
0224    font-size: 10px;
0225    font-weight: 700;
0226  }
0227
0228  .profile strong,.profile small {
0229    display: block;
0230  }
0231
0232  .profile strong {
0233    font-size: 11px;
0234    font-weight: 600;
0235  }
0236
0237  .profile small {
0238    font-size: 10px;
0239    color: #9295aa;
0240    margin-top: 4px;
0241  }
0242
0243  .profile-dots {
0244    margin-left: auto;
0245    color: #a3a5b4;
0246    font-size: 20px;
0247  }
0248
```

styles.css - lines 249-310

```
0249  .main-area {
0250    margin-left: 244px;
0251    min-height: 100vh;
0252  }
0253
0254  .topbar {
0255    height: 68px;
0256    border-bottom: 1px solid #e9eaf0;
0257    display: flex;
0258    align-items: center;
0259    justify-content: space-between;
0260    padding: 0 44px;
0261    background: rgba(245,247,250,.9);
0262  }
0263
0264  .breadcrumb {
0265    display: flex;
0266    align-items: center;
0267    gap: 11px;
0268    color: #9b9eaa;
0269    font-size: 11px;
0270  }
0271
0272  .breadcrumb b {
0273    font-weight: 400;
0274    color: #c5c7d0;
0275  }
0276
0277  .breadcrumb strong {
0278    color: #4a4c5e;
0279    font-weight: 600;
0280  }
0281
0282  .topbar-right {
0283    display: flex;
0284    align-items: center;
0285    gap: 16px;
0286  }
0287
0288  .today-chip {
0289    font-size: 11px;
0290    color: #65687b;
0291    display: flex;
0292    gap: 8px;
0293    align-items: center;
0294  }
0295
0296  .live-dot,.mini-dot {
0297    width: 7px;
0298    height: 7px;
0299    border-radius: 50%;
0300    background: #83ca72;
0301    display: inline-block;
0302  }
0303
0304  .icon-button,.mobile-menu {
0305    background: none;
0306    border: 0;
0307    color: #8b8ea0;
0308    font-size: 19px;
0309  }
0310
```

styles.css - lines 311-372

```
.icon-button:hover {
  color: var(--midnight);
}

.avatar-small {
  width: 28px;
  height: 28px;
  font-size: 9px;
}

.mobile-menu {
  display: none;
}

.content-wrap {
  max-width: 1270px;
  padding: 37px 44px 20px;
  margin: auto;
}

.welcome-row {
  display: flex;
  align-items: flex-end;
  justify-content: space-between;
  margin-bottom: 27px;
}

.eyebrow {
  color: #9a9dab;
}

.welcome-row h1 {
  font: 700 29px/1.2 Manrope,sans-serif;
  letter-spacing: -1.1px;
  margin: 9px 0 7px;
  color: var(--midnight);
}

.welcome-row h1 span {
  color: #7dbb6c;
}

.page-subtitle {
  color: #8b8e9c;
  margin: 0;
  font-size: 12px;
  line-height: 1.7;
}

.button {
  height: 39px;
  border: 0;
  border-radius: 8px;
  padding: 0 15px;
  font-size: 12px;
  font-weight: 600;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  transition: .2s;
}
```

styles.css - lines 373-434

```
0373
0374  .button-dark {
0375    background: var(--midnight);
0376    color: #fff;
0377  }
0378
0379  .button-dark:hover {
0380    background: #30314e;
0381    transform: translateY(-1px);
0382  }
0383
0384  .button-dark span {
0385    color: var(--green);
0386    font-size: 17px;
0387  }
0388
0389  .button-quiet {
0390    background: #f2f3f7;
0391    color: #55586c;
0392  }
0393
0394  .page-section {
0395    display: none;
0396    animation: appear .25s ease;
0397  }
0398
0399  .page-section.active-page {
0400    display: block;
0401  }
0402
0403  @keyframes appear {
0404    from {
0405      opacity: .4;
0406      transform: translateY(4px);
0407    }
0408
0409    to {
0410      opacity: 1;
0411      transform: none;
0412    }
0413
0414  }
0415
0416  /* Dashboard summary cards */
0417
0418  .stats-grid {
0419    display: grid;
0420    grid-template-columns: repeat(3,1fr);
0421    gap: 15px;
0422    margin-bottom: 18px;
0423  }
0424
0425  .stat-card,.panel {
0426    background: var(--white);
0427    border: 1px solid #eceef3;
0428    border-radius: 12px;
0429    box-shadow: var(--shadow);
0430  }
0431
0432  .stat-card {
0433    padding: 17px 19px;
0434  }
```

styles.css - lines 435-496

```
0435
0436  .stat-top {
0437    display: flex;
0438    justify-content: space-between;
0439    align-items: center;
0440    color: #797c8c;
0441    font-size: 11px;
0442  }
0443
0444  .stat-icon {
0445    width: 29px;
0446    height: 29px;
0447    display: grid;
0448    place-items: center;
0449    border-radius: 8px;
0450    font-size: 15px;
0451  }
0452
0453  .icon-green {
0454    background: #eaf7e5;
0455    color: #73b765;
0456  }
0457
0458  .icon-peach {
0459    background: #fff0e5;
0460    color: #d68f63;
0461  }
0462
0463  .icon-lilac {
0464    background: #f0eafb;
0465    color: #987cc8;
0466  }
0467
0468  .stat-number {
0469    font: 700 25px Manrope,sans-serif;
0470    letter-spacing: -.8px;
0471    color: var(--midnight);
0472    margin: 8px 0 7px;
0473  }
0474
0475  .stat-number small {
0476    font: 500 11px 'DM Sans',sans-serif;
0477    letter-spacing: 0;
0478    color: #9396a3;
0479  }
0480
0481  .progress-track {
0482    height: 4px;
0483    background: #edf0ea;
0484    border-radius: 4px;
0485    overflow: hidden;
0486    margin: 11px 0 8px;
0487  }
0488
0489  .progress-track span,.budget-bar span {
0490    height: 100%;
0491    display: block;
0492    background: var(--green);
0493    width: 0;
0494    transition: width .3s;
0495    border-radius: 4px;
0496  }
```

styles.css - lines 497-558

```
0497
0498  .stat-foot {
0499    font-size: 10px;
0500    color: #a0a2ad;
0501    margin-top: 8px;
0502    display: flex;
0503    align-items: center;
0504    gap: 6px;
0505  }
0506
0507  .dashboard-grid {
0508    display: grid;
0509    grid-template-columns: 1.1fr .9fr;
0510    gap: 16px;
0511  }
0512
0513  .panel {
0514    padding: 19px;
0515  }
0516
0517  .panel-heading {
0518    display: flex;
0519    justify-content: space-between;
0520    align-items: center;
0521    margin-bottom: 17px;
0522  }
0523
0524  .panel-heading h2,.planner-side h2 {
0525    font: 700 16px Manrope,sans-serif;
0526    letter-spacing: -.35px;
0527    color: var(--midnight);
0528    margin: 6px 0 0;
0529  }
0530
0531  .panel-heading .eyebrow {
0532    font-size: 9px;
0533  }
0534
0535  .heading-count {
0536    display: inline-grid;
0537    place-items: center;
0538    background: #f0f1f5;
0539    border-radius: 20px;
0540    min-width: 21px;
0541    height: 21px;
0542    padding: 0 7px;
0543    font-size: 10px;
0544    color: #787b89;
0545    margin-left: 4px;
0546    vertical-align: 2px;
0547  }
0548
0549  .text-link {
0550    font-size: 10px;
0551    color: #878a99;
0552    text-decoration: none;
0553  }
0554
0555  .text-link span {
0556    color: #7ebc6e;
0557    margin-left: 4px;
0558    font-size: 14px;
```

styles.css - lines 559-620

```
0559  }
0560
0561  .text-link:hover {
0562    color: var(--midnight);
0563  }
0564
0565  /* Reusable task rows and schedule cards */
0566
0567  .task-list {
0568    display: grid;
0569  }
0570
0571  .task-row {
0572    display: flex;
0573    align-items: center;
0574    gap: 11px;
0575    min-height: 49px;
0576    border-bottom: 1px solid #f0f1f5;
0577    padding: 7px 0;
0578  }
0579
0580  .task-row:last-child {
0581    border-bottom: 0;
0582  }
0583
0584  .check-button {
0585    width: 17px;
0586    height: 17px;
0587    flex: none;
0588    border-radius: 5px;
0589    border: 1.4px solid #d3d6df;
0590    background: #fff;
0591    color: var(--midnight);
0592    display: grid;
0593    place-items: center;
0594    font-size: 11px;
0595    padding: 0;
0596  }
0597
0598  .check-button.checked {
0599    background: var(--green);
0600    border-color: var(--green);
0601  }
0602
0603  .task-copy {
0604    min-width: 0;
0605    flex: 1;
0606  }
0607
0608  .task-copy strong {
0609    display: block;
0610    font-size: 11px;
0611    font-weight: 600;
0612    color: #4d5061;
0613    white-space: nowrap;
0614    overflow: hidden;
0615    text-overflow: ellipsis;
0616  }
0617
0618  .task-copy small {
0619    display: block;
0620    font-size: 9px;
```

styles.css - lines 621-682

```
0621    color: #a4a6b1;
0622    margin-top: 4px;
0623  }
0624
0625  .task-done .task-copy strong {
0626    text-decoration: line-through;
0627    color: #aaacb6;
0628  }
0629
0630  .priority {
0631    font-size: 9px;
0632    padding: 4px 7px;
0633    border-radius: 12px;
0634    white-space: nowrap;
0635  }
0636
0637  .priority.high {
0638    background: #fff0ec;
0639    color: #d77b64;
0640  }
0641
0642  .priority.medium {
0643    background: #fff6e7;
0644    color: #c79c56;
0645  }
0646
0647  .priority.low {
0648    background: #eff6ec;
0649    color: #76a56c;
0650  }
0651
0652  .task-actions {
0653    opacity: 0;
0654    display: flex;
0655    gap: 2px;
0656  }
0657
0658  .task-row:hover .task-actions,.task-row:focus-within .task-actions {
0659    opacity: 1;
0660  }
0661
0662  .tiny-action {
0663    border: 0;
0664    background: none;
0665    color: #a0a2ac;
0666    padding: 3px;
0667    font-size: 13px;
0668  }
0669
0670  .tiny-action:hover {
0671    color: #b65b55;
0672  }
0673
0674  .add-inline {
0675    margin-top: 11px;
0676    border: 0;
0677    background: none;
0678    padding: 6px 0;
0679    color: #8d909d;
0680    font-size: 10px;
0681  }
0682
```

styles.css - lines 683-744

```
0683  .add-inline:hover {
0684    color: var(--midnight);
0685  }
0686
0687  .next-class {
0688    display: grid;
0689    gap: 11px;
0690  }
0691
0692  .class-card {
0693    display: flex;
0694    align-items: center;
0695    gap: 12px;
0696    padding: 11px;
0697    background: #f8f9fb;
0698    border-radius: 8px;
0699  }
0700
0701  .class-time {
0702    width: 53px;
0703    font-size: 10px;
0704    color: #787b8b;
0705    font-weight: 600;
0706  }
0707
0708  .class-stripe {
0709    width: 3px;
0710    height: 34px;
0711    border-radius: 3px;
0712    background: #90d67c;
0713  }
0714
0715  .class-info {
0716    flex: 1;
0717  }
0718
0719  .class-info strong {
0720    display: block;
0721    font-size: 11px;
0722    color: #46495b;
0723  }
0724
0725  .class-info small {
0726    display: block;
0727    margin-top: 4px;
0728    font-size: 9px;
0729    color: #9699a7;
0730  }
0731
0732  .class-room {
0733    color: #999ba8;
0734    font-size: 9px;
0735  }
0736
0737  .no-class {
0738    padding: 20px 0;
0739    color: #9295a3;
0740    font-size: 11px;
0741  }
0742
0743  .spend-total {
0744    display: flex;
```

styles.css - lines 745-806

```
0745    align-items: baseline;
0746    gap: 8px;
0747  }
0748
0749  .spend-total strong {
0750    font: 700 24px Manrope,sans-serif;
0751    color: var(--midnight);
0752  }
0753
0754  .spend-total span {
0755    font-size: 10px;
0756    color: #999ba7;
0757  }
0758
0759  .budget-bar {
0760    height: 7px;
0761    background: #eff1f0;
0762    border-radius: 6px;
0763    overflow: hidden;
0764    margin: 16px 0 7px;
0765  }
0766
0767  .budget-bar span {
0768    background: #91d87d;
0769  }
0770
0771  .budget-labels {
0772    display: flex;
0773    justify-content: space-between;
0774    color: #a0a2ac;
0775    font-size: 9px;
0776  }
0777
0778  .recent-transactions {
0779    margin-top: 14px;
0780  }
0781
0782  .transaction-row {
0783    display: flex;
0784    align-items: center;
0785    gap: 9px;
0786    padding: 9px 0;
0787    border-top: 1px solid #f0f1f5;
0788  }
0789
0790  .transaction-icon {
0791    width: 27px;
0792    height: 27px;
0793    display: grid;
0794    place-items: center;
0795    background: #f3f4f7;
0796    border-radius: 8px;
0797    color: #828596;
0798  }
0799
0800  .transaction-info {
0801    flex: 1;
0802  }
0803
0804  .transaction-info strong {
0805    font-size: 10px;
0806    font-weight: 600;
```

styles.css - lines 807-868

```
0807    color: #585b6a;
0808    display: block;
0809  }
0810
0811  .transaction-info small {
0812    font-size: 9px;
0813    color: #a0a2ad;
0814    display: block;
0815    margin-top: 3px;
0816  }
0817
0818  .transaction-amount {
0819    font-size: 10px;
0820    font-weight: 600;
0821    color: #55586b;
0822  }
0823
0824  .focus-panel {
0825    background: var(--midnight);
0826    color: #fff;
0827    position: relative;
0828    overflow: hidden;
0829    min-height: 177px;
0830    display: flex;
0831    align-items: center;
0832    padding-left: 23px;
0833  }
0834
0835  .focus-copy {
0836    position: relative;
0837    z-index: 1;
0838    max-width: 75%;
0839  }
0840
0841  .focus-badge {
0842    font-size: 8px;
0843    letter-spacing: 1.2px;
0844    font-weight: 700;
0845    color: var(--green);
0846  }
0847
0848  .focus-copy h2 {
0849    font: 600 16px/1.5 Manrope,sans-serif;
0850    letter-spacing: -.3px;
0851    margin: 11px 0 7px;
0852    color: #fff;
0853  }
0854
0855  .focus-copy h2 em {
0856    font-style: normal;
0857    color: var(--green);
0858  }
0859
0860  .focus-copy p {
0861    font-size: 10px;
0862    color: #a3a6ba;
0863    margin: 0;
0864  }
0865
0866  .focus-art {
0867    position: absolute;
0868    right: -10px;
```

styles.css - lines 869-930

```
0869    bottom: -27px;
0870    width: 148px;
0871    height: 148px;
0872    border-radius: 50%;
0873    background: #242642;
0874  }
0875
0876  .art-sun {
0877    position: absolute;
0878    top: 19px;
0879    right: 22px;
0880    width: 41px;
0881    height: 41px;
0882    border-radius: 50%;
0883    background: var(--green);
0884  }
0885
0886  .art-star {
0887    position: absolute;
0888    color: #d7b8fc;
0889    font-size: 21px;
0890    left: 26px;
0891    top: 20px;
0892  }
0893
0894  .art-plant {
0895    position: absolute;
0896    bottom: 7px;
0897    right: 59px;
0898    color: #a3aa84;
0899    font-size: 65px;
0900    transform: rotate(-20deg);
0901  }
0902
0903  .footer-note {
0904    padding: 28px 0 6px;
0905    text-align: center;
0906    color: #a4a6b1;
0907    font-size: 9px;
0908  }
0909
0910  .footer-note span {
0911    color: #85c875;
0912    margin-left: 4px;
0913  }
0914
0915  /* Planner, timetable, budget, and grade page layouts */
0916
0917  .planner-layout {
0918    display: grid;
0919    grid-template-columns: 1.55fr .75fr;
0920    gap: 16px;
0921  }
0922
0923  .planner-main {
0924    padding: 22px;
0925  }
0926
0927  .planner-side {
0928    padding: 22px;
0929    background: #fff;
0930  }
```

styles.css - lines 931-992

```
0931
0932  .filter-tabs {
0933    display: flex;
0934    gap: 5px;
0935    border-bottom: 1px solid var(--line);
0936    margin-bottom: 8px;
0937  }
0938
0939  .filter-tab {
0940    border: 0;
0941    background: none;
0942    color: #9699a6;
0943    padding: 9px 11px;
0944    font-size: 10px;
0945    border-bottom: 2px solid transparent;
0946  }
0947
0948  .filter-tab.selected {
0949    color: var(--midnight);
0950    border-color: var(--green);
0951    font-weight: 600;
0952  }
0953
0954  .full-task-list .task-row {
0955    min-height: 58px;
0956  }
0957
0958  .planner-side h2 {
0959    font-size: 18px;
0960    margin: 12px 0 8px;
0961  }
0962
0963  .planner-side .page-subtitle {
0964    font-size: 11px;
0965  }
0966
0967  .break-card {
0968    display: flex;
0969    align-items: center;
0970    gap: 12px;
0971    background: #f5f8f1;
0972    border-radius: 9px;
0973    padding: 14px;
0974    margin-top: 22px;
0975  }
0976
0977  .break-card>span {
0978    color: #7cbd6e;
0979    font-size: 20px;
0980  }
0981
0982  .break-card strong,.break-card small {
0983    display: block;
0984  }
0985
0986  .break-card strong {
0987    font-size: 11px;
0988    color: #535668;
0989  }
0990
0991  .break-card small {
0992    font-size: 9px;
```

styles.css - lines 993-1054

```
0993    color: #9b9eaa;
0994    margin-top: 4px;
0995  }
0996
0997  .week-mini {
0998    margin-top: 24px;
0999    display: grid;
1000    grid-template-columns: repeat(7,1fr);
1001    gap: 4px;
1002  }
1003
1004  .mini-day {
1005    text-align: center;
1006    font-size: 8px;
1007    color: #a4a6b2;
1008  }
1009
1010  .mini-day b {
1011    display: grid;
1012    place-items: center;
1013    height: 26px;
1014    width: 26px;
1015    margin: 7px auto 0;
1016    border-radius: 50%;
1017    font-size: 9px;
1018    font-weight: 500;
1019    color: #707384;
1020  }
1021
1022  .mini-day.current b {
1023    background: var(--green);
1024    color: var(--midnight);
1025    font-weight: 700;
1026  }
1027
1028  .timetable-panel {
1029    padding: 22px;
1030  }
1031
1032  .schedule-legend {
1033    font-size: 10px;
1034    color: #9295a3;
1035  }
1036
1037  .schedule-legend span {
1038    display: flex;
1039    align-items: center;
1040    gap: 6px;
1041  }
1042
1043  .legend-dot {
1044    width: 7px;
1045    height: 7px;
1046    border-radius: 50%;
1047    background: var(--green);
1048  }
1049
1050  .table-scroll {
1051    overflow-x: auto;
1052  }
1053
1054  .timetable {
```

styles.css - lines 1055-1116

```
1055    min-width: 720px;
1056    display: grid;
1057    grid-template-columns: 72px repeat(5,minmax(0,1fr));
1058    border: 1px solid #eff0f4;
1059    border-radius: 8px;
1060    overflow: hidden;
1061  }
1062
1063  .time-head,.day-head {
1064    background: #fafbfc;
1065    padding: 13px 9px;
1066    font-size: 9px;
1067    font-weight: 700;
1068    letter-spacing: .5px;
1069    color: #8e91a0;
1070    text-align: center;
1071    border-bottom: 1px solid #eff0f4;
1072  }
1073
1074  .time-head {
1075    border-right: 1px solid #eff0f4;
1076  }
1077
1078  .time-slot {
1079    min-height: 66px;
1080    padding: 8px 6px;
1081    text-align: center;
1082    font-size: 9px;
1083    color: #a4a6b1;
1084    border-right: 1px solid #f0f1f5;
1085    border-bottom: 1px solid #f0f1f5;
1086  }
1087
1088  .class-slot {
1089    min-height: 66px;
1090    padding: 5px;
1091    display: flex;
1092    flex-direction: column;
1093    gap: 5px;
1094    border-bottom: 1px solid #f0f1f5;
1095    border-right: 1px solid #f0f1f5;
1096    position: relative;
1097  }
1098
1099  .class-block {
1100    min-height: 50px;
1101    border-radius: 6px;
1102    background: #edf8e9;
1103    border-left: 3px solid #8DDA7A;
1104    padding: 7px 7px;
1105    color: #4e6b48;
1106    cursor: pointer;
1107  }
1108
1109  .class-block:hover {
1110    background: #e1f2db;
1111  }
1112
1113  .class-block strong {
1114    font-size: 9px;
1115    display: block;
1116    line-height: 1.35;
```

styles.css - lines 1117-1178

```
}

.class-block small {
  font-size: 8px;
  display: block;
  color: #799474;
  margin-top: 4px;
}

.section-hint {
  font-size: 10px;
  color: #9b9daa;
  margin: 13px 3px;
}

.budget-stats {
  display: grid;
  grid-template-columns: repeat(3,1fr);
  gap: 14px;
  margin-bottom: 17px;
}

.budget-stat {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 10px;
}

.budget-stat>span {
  font-size: 10px;
  color: #9295a3;
}

.budget-stat>strong {
  font: 700 23px Manrope,sans-serif;
  letter-spacing: -.7px;
  color: var(--midnight);
}

.budget-stat small {
  font-size: 10px;
  color: #a2a4ae;
}

.budget-rule-panel {
  padding: 21px;
  margin-bottom: 17px;
}

.rule-total {
  color: var(--slate);
  font-size: 10px;
}

.rule-total strong {
  color: var(--midnight);
  font: 700 13px Manrope,sans-serif;
}

.rule-intro {
  margin: -4px 0 14px;
```

styles.css - lines 1179-1240

```
1179    color: var(--slate);
1180    font-size: 11px;
1181  }
1182
1183  .budget-rule-grid {
1184    display: grid;
1185    grid-template-columns: repeat(3,minmax(0,1fr));
1186    gap: 12px;
1187  }
1188
1189  .rule-card {
1190    display: flex;
1191    flex-direction: column;
1192    gap: 8px;
1193    padding: 15px;
1194    border-radius: 9px;
1195    background: var(--cloud);
1196  }
1197
1198  .rule-card span {
1199    color: var(--slate);
1200    font-size: 9px;
1201    font-weight: 700;
1202    letter-spacing: .6px;
1203  }
1204
1205  .rule-card strong {
1206    color: var(--midnight);
1207    font: 700 21px Manrope,sans-serif;
1208  }
1209
1210  .rule-card small {
1211    color: var(--slate);
1212    font-size: 9px;
1213    line-height: 1.5;
1214  }
1215
1216  .rule-needs {
1217    background: #f0f2fa;
1218  }
1219
1220  .rule-wants {
1221    background: #f5f2fa;
1222  }
1223
1224  .rule-savings {
1225    background: #edf8e9;
1226  }
1227
1228  .rule-note {
1229    margin: 12px 2px 0;
1230    color: var(--slate);
1231    font-size: 9px;
1232  }
1233
1234  .subtle-button {
1235    border: 0;
1236    background: none;
1237    color: #7eae72;
1238    padding: 0;
1239    font-size: 10px;
1240  }
```

styles.css - lines 1241-1302

```
1241
1242  .expenses-panel {
1243    padding: 21px;
1244  }
1245
1246  .select-small {
1247    border: 1px solid #eceef3;
1248    border-radius: 7px;
1249    padding: 8px 10px;
1250    background: white;
1251    color: #777a89;
1252    font-size: 10px;
1253    outline: none;
1254  }
1255
1256  .expense-row {
1257    display: grid;
1258    grid-template-columns: 35px 1fr auto auto;
1259    gap: 12px;
1260    align-items: center;
1261    border-bottom: 1px solid #f0f1f5;
1262    padding: 12px 1px;
1263  }
1264
1265  .expense-row:last-child {
1266    border: 0;
1267  }
1268
1269  .expense-icon {
1270    width: 31px;
1271    height: 31px;
1272    border-radius: 9px;
1273    background: #f2f3f6;
1274    display: grid;
1275    place-items: center;
1276    font-size: 14px;
1277  }
1278
1279  .expense-copy strong,.expense-copy small {
1280    display: block;
1281  }
1282
1283  .expense-copy strong {
1284    font-size: 11px;
1285    color: #545768;
1286  }
1287
1288  .expense-copy small {
1289    font-size: 9px;
1290    color: #a1a3ae;
1291    margin-top: 4px;
1292  }
1293
1294  .expense-category {
1295    font-size: 9px;
1296    color: #9093a0;
1297    background: #f7f8fa;
1298    border-radius: 12px;
1299    padding: 5px 8px;
1300  }
1301
1302  .expense-amount {
```

styles.css - lines 1303-1364

```
  font-size: 11px;
  font-weight: 600;
  color: #4f5263;
  min-width: 60px;
  text-align: right;
}

.grade-summary {
  border-radius: 12px;
  background: var(--midnight);
  min-height: 150px;
  color: white;
  padding: 24px 29px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 17px;
  overflow: hidden;
  position: relative;
}

.grade-summary .eyebrow {
  color: #a3a5b8;
}

.grade-big {
  font: 700 39px Manrope,sans-serif;
  letter-spacing: -1.5px;
  color: var(--green);
  margin: 6px 0 2px;
}

.grade-big small {
  font-size: 15px;
  margin-left: 3px;
}

.grade-caption {
  margin: 0;
  font-size: 10px;
  color: #b0b2c2;
}

.grade-summary-art {
  width: 135px;
  height: 135px;
  background: #242642;
  border-radius: 50%;
  margin: -40px 20px -60px 0;
  display: grid;
  place-items: center;
  color: #b8a3e2;
  font-size: 47px;
}

.grades-panel {
  padding: 22px;
}

.grade-weights-panel {
  padding: 22px;
  margin-bottom: 17px;
```

styles.css - lines 1365-1426

```
}

.grade-weight-total {
  padding: 6px 9px;
  border-radius: 20px;
  background: var(--cloud);
  color: var(--midnight);
  font-size: 10px;
  font-weight: 700;
  white-space: nowrap;
}

.grade-weight-help {
  margin: -4px 0 14px;
  color: var(--slate);
  font-size: 10px;
  line-height: 1.5;
}

.grade-weight-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit,minmax(125px,1fr));
  gap: 10px;
}

.grade-weight-card {
  padding: 10px;
  border: 1px solid #eceef3;
  border-radius: 8px;
  background: #fff;
}

.grade-weight-card label {
  display: block;
  margin-bottom: 7px;
  color: #646778;
  font-size: 9px;
  font-weight: 600;
}

.grade-weight-input-wrap {
  display: flex;
  align-items: center;
  gap: 6px;
  color: var(--slate);
}

.grade-weight-input-wrap input {
  width: 100%;
  min-width: 0;
  padding: 7px 8px;
  border: 1px solid #e8eaf0;
  border-radius: 6px;
  color: var(--midnight);
  font: 600 12px Manrope,sans-serif;
}

.grade-weight-input-wrap input:focus {
  outline: 2px solid #8dda7a66;
  border-color: var(--green);
}

```

styles.css - lines 1427-1488

```
.grade-weight-status {
  margin: 11px 1px 0;
  color: #a45a48;
  font-size: 9px;
}

.grade-weight-status.is-valid {
  color: #56844b;
}

.grade-count {
  font-size: 10px;
  color: #999ba8;
}

.grades-table {
  border-collapse: collapse;
  width: 100%;
  min-width: 550px;
}

.grades-table th {
  text-align: left;
  padding: 10px 8px;
  border-bottom: 1px solid #eff0f4;
  color: #a2a4af;
  font-size: 8px;
  letter-spacing: 1px;
  font-weight: 700;
}

.grades-table td {
  padding: 13px 8px;
  border-bottom: 1px solid #f0f1f5;
  color: #646778;
  font-size: 10px;
}

.grades-table tr:last-child td {
  border-bottom: 0;
}

.grades-table td:first-child {
  font-weight: 600;
  color: #4c4f60;
}

.grades-table input {
  width: 76px;
  padding: 6px 7px;
  border: 1px solid #e8eaf0;
  border-radius: 6px;
  color: #4d5060;
  font-size: 10px;
  outline: none;
}

.grades-table select {
  width: 100%;
  min-width: 112px;
  padding: 6px 7px;
  border: 1px solid #e8eaf0;
```

styles.css - lines 1489-1550

```
  border-radius: 6px;
  background: #fff;
  color: #4d5060;
  font-size: 10px;
  outline: none;
}

.grades-table input:focus {
  border-color: #8DDA7A;
}

.grades-table select:focus {
  border-color: #8DDA7A;
}

.grade-raw {
  color: #4f5263!important;
  font-weight: 600;
  white-space: nowrap;
}

.grade-actions {
  display: flex;
  justify-content: center;
  gap: 10px;
  margin: 18px 0 4px;
}

.row-action {
  border: 0;
  background: transparent;
  color: #a2a4af;
  font-size: 13px;
}

.row-action:hover {
  color: #cb7164;
}

.grade-note {
  font-size: 9px;
  color: #a0a2ac;
  margin: 12px 7px 0;
}

.empty-state {
  text-align: center;
  color: #a1a3ae;
  padding: 25px 10px;
  font-size: 11px;
}

/* Entry dialog, form fields, and feedback toast */

.dialog-head {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  margin-bottom: 20px;
}

.dialog-head h2 {
```

styles.css - lines 1551-1612

```
1551    font: 700 20px Manrope,sans-serif;
1552    color: var(--midnight);
1553    margin: 7px 0 0;
1554    letter-spacing: -.5px;
1555  }
1556
1557  .close-dialog {
1558    border: 0;
1559    background: none;
1560    color: #9295a3;
1561    font-size: 22px;
1562    line-height: 1;
1563  }
1564
1565  .dialog-field {
1566    margin-bottom: 13px;
1567  }
1568
1569  .dialog-field label {
1570    display: block;
1571    color: #777a89;
1572    font-size: 10px;
1573    font-weight: 600;
1574    margin-bottom: 6px;
1575  }
1576
1577  .dialog-field input,.dialog-field select {
1578    width: 100%;
1579    height: 38px;
1580    border: 1px solid #e8eaf0;
1581    border-radius: 7px;
1582    padding: 0 10px;
1583    color: #4b4e5f;
1584    background: #fff;
1585    outline: none;
1586    font-size: 11px;
1587  }
1588
1589  .dialog-field input:focus,.dialog-field select:focus {
1590    border-color: #8DDA7A;
1591    box-shadow: 0 0 0 3px #8dda7a25;
1592  }
1593
1594  .dialog-grid {
1595    display: grid;
1596    grid-template-columns: 1fr 1fr;
1597    gap: 11px;
1598  }
1599
1600  .dialog-actions {
1601    display: flex;
1602    justify-content: flex-end;
1603    gap: 8px;
1604    margin-top: 20px;
1605  }
1606
1607  .dialog-actions .button {
1608    height: 36px;
1609  }
1610
1611  dialog {
1612    width: min(420px,calc(100% - 30px));
```

styles.css - lines 1613-1674

```
  border: 1px solid #eceef3;
  border-radius: 13px;
  padding: 23px 24px;
  box-shadow: 0 25px 80px #17182f35;
}

dialog::backdrop {
  background: #17182f65;
  backdrop-filter: blur(2px);
}

.toast {
  position: fixed;
  bottom: 23px;
  left: 50%;
  transform: translate(-50%,15px);
  padding: 10px 15px;
  border-radius: 8px;
  background: var(--midnight);
  color: white;
  font-size: 11px;
  opacity: 0;
  pointer-events: none;
  transition: .22s;
  z-index: 40;
  box-shadow: 0 6px 20px #17182f30;
}

.toast.show {
  opacity: 1;
  transform: translate(-50%,0);
}

/* Tablet adjustments */

@media(max-width:1050px) {
  .content-wrap {
    padding: 30px 27px 18px;
  }

  .topbar {
    padding: 0 27px;
  }

  .sidebar {
    width: 220px;
  }

  .main-area {
    margin-left: 220px;
  }

  .dashboard-grid {
    gap: 12px;
  }

  .focus-copy {
    max-width: 82%;
  }

}

```

styles.css - lines 1675-1736

```
@media(max-width:800px) {
  .sidebar {
    transform: translateX(-100%);
    transition: transform .25s;
    width: 245px;
    box-shadow: 10px 0 40px #17182f22;
  }

  .sidebar.open {
    transform: translateX(0);
  }

  .main-area {
    margin-left: 0;
  }

  .mobile-menu {
    display: block;
    padding: 4px 8px 4px 0;
  }

  .topbar {
    padding: 0 20px;
  }

  .breadcrumb {
    margin-right: auto;
  }

  .content-wrap {
    padding: 27px 20px 18px;
  }

  /* Tablet planner and timetable layout */

  .planner-layout {
    grid-template-columns: 1fr .72fr;
  }

  .dashboard-grid {
    grid-template-columns: 1fr 1fr;
  }

  .welcome-row h1 {
    font-size: 26px;
  }

}

/* Phone adjustments */

@media(max-width:600px) {
  .topbar {
    height: 59px;
    padding: 0 16px;
  }

  .topbar-right {
    gap: 10px;
  }

  .today-chip {
```

styles.css - lines 1737-1798

```
    font-size: 0;
  }

  .today-chip .live-dot {
    font-size: initial;
  }

  .content-wrap {
    padding: 24px 15px 15px;
  }

  .welcome-row {
    align-items: flex-start;
    gap: 12px;
    margin-bottom: 21px;
  }

  .welcome-row h1 {
    font-size: 23px;
  }

  .page-subtitle {
    font-size: 11px;
    max-width: 250px;
  }

  .welcome-row>.button {
    white-space: nowrap;
    padding: 0 10px;
    height: 36px;
    font-size: 10px;
    margin-top: 11px;
  }

  /* Compact dashboard summary cards */

  .stats-grid {
    grid-template-columns: 1fr;
    gap: 9px;
  }

  .stat-card {
    padding: 14px 16px;
  }

  .stat-number {
    font-size: 22px;
    margin: 5px 0;
  }

  .stat-icon {
    width: 27px;
    height: 27px;
  }

  .stat-foot {
    margin-top: 5px;
  }

  .progress-track {
    margin: 8px 0 5px;
  }
```

styles.css - lines 1799-1860

```
1799
1800    .dashboard-grid {
1801      grid-template-columns: 1fr;
1802      gap: 10px;
1803    }
1804
1805    .panel {
1806      padding: 16px;
1807    }
1808
1809    .focus-panel {
1810      min-height: 160px;
1811    }
1812
1813    /* Phone planner, budget, and grade layout */
1814
1815    .planner-layout {
1816      grid-template-columns: 1fr;
1817    }
1818
1819    .planner-side {
1820      order: -1;
1821    }
1822
1823    .planner-side .week-mini {
1824      margin-top: 15px;
1825    }
1826
1827    .budget-stats {
1828      grid-template-columns: 1fr;
1829      gap: 9px;
1830    }
1831
1832    .budget-rule-grid {
1833      grid-template-columns: 1fr;
1834      gap: 8px;
1835    }
1836
1837    .budget-stat {
1838      display: grid;
1839      grid-template-columns: 1fr auto;
1840      gap: 7px 10px;
1841      align-items: center;
1842    }
1843
1844    .budget-stat>span {
1845      grid-column: 1;
1846    }
1847
1848    .budget-stat>strong {
1849      grid-column: 1;
1850      grid-row: 2;
1851    }
1852
1853    .budget-stat .subtle-button,.budget-stat small {
1854      grid-column: 2;
1855      grid-row: 2;
1856    }
1857
1858    .expenses-panel {
1859      padding: 15px;
1860    }
```

styles.css - lines 1861-1917

```
1861
1862    .expense-row {
1863      grid-template-columns: 32px 1fr auto;
1864      gap: 8px;
1865    }
1866
1867    .expense-category {
1868      display: none;
1869    }
1870
1871    .expense-amount {
1872      min-width: 54px;
1873    }
1874
1875    .grade-summary {
1876      padding: 20px;
1877      min-height: 135px;
1878    }
1879
1880    .grade-summary-art {
1881      margin-right: -35px;
1882    }
1883
1884    .grades-panel {
1885      padding: 16px;
1886    }
1887
1888    .grade-weights-panel {
1889      padding: 16px;
1890    }
1891
1892    .grade-weight-grid {
1893      grid-template-columns: repeat(2,minmax(0,1fr));
1894    }
1895
1896    .grades-table-wrap {
1897      overflow-x: auto;
1898    }
1899
1900    .footer-note {
1901      line-height: 1.7;
1902    }
1903
1904    .dialog-grid {
1905      grid-template-columns: 1fr 1fr;
1906    }
1907
1908  }
1909
1910  @media(prefers-reduced-motion:reduce) {
1911    *,*:before,*:after {
1912      scroll-behavior: auto!important;
1913      animation-duration: .01ms!important;
1914      transition-duration: .01ms!important;
1915    }
1916
1917  }
```

Appendix - C - JavaScript behavior

Source file: app.js. The listing below is read from the project file when this guide is generated. It has 764 source lines.

Visual line wraps may split a long source line across several printed rows. The numbered source line remains one line in the editable file.

app.js - lines 1-62

```
0001  (() => {
0002    // 1. Starter data and settings
0003    // These values set the first-run sample content and the timetable/grade options.
0004    const KEY = "plannex-student-planner-v1";
0005    const DAYS = ["Mon", "Tue", "Wed", "Thu", "Fri"];
0006    const HOURS = [
0007      "08:00",
0008      "09:00",
0009      "10:00",
0010      "11:00",
0011      "12:00",
0012      "13:00",
0013      "14:00",
0014      "15:00",
0015      "16:00",
0016    ];
0017    const GRADE_CATEGORIES = ["Exam", "Performance Task", "Quiz", "Long Quiz", "Assignment"];
0018    const seed = {
0019      allowance: 12000,
0020      gradeWeights: {
0021        Exam: 40,
0022        "Performance Task": 40,
0023        Quiz: 10,
0024        "Long Quiz": 0,
0025        Assignment: 10,
0026      },
0027      tasks: [
0028        {
0029          id: "t1",
0030          title: "Finish biology lab notes",
0031          due: "Today · 11:30 AM",
0032          priority: "High",
0033          done: false,
0034        },
0035        {
0036          id: "t2",
0037          title: "Read chapter 4 for literature",
0038          due: "Today · 2:00 PM",
0039          priority: "Medium",
0040          done: false,
0041        },
0042        {
0043          id: "t3",
0044          title: "Submit statistics problem set",
0045          due: "Today · 5:00 PM",
0046          priority: "High",
0047          done: true,
0048        },
0049        {
0050          id: "t4",
0051          title: "Pack things for tomorrow",
0052          due: "Today · Anytime",
0053          priority: "Low",
0054          done: false,
0055        },
0056      ],
0057      classes: [
0058        {
0059          id: "c1",
0060          day: "Mon",
0061          time: "9:00",
0062          name: "Biology 101",
```

app.js - lines 63-124

```
0063          room: "Science Hall · 204",
0064        },
0065        {
0066          id: "c2",
0067          day: "Mon",
0068          time: "11:00",
0069          name: "Literature & Society",
0070          room: "Arts Building · 12",
0071        },
0072        {
0073          id: "c3",
0074          day: "Mon",
0075          time: "2:00",
0076          name: "Statistics",
0077          room: "Main Building · 305",
0078        },
0079        {
0080          id: "c4",
0081          day: "Tue",
0082          time: "8:00",
0083          name: "Communication",
0084          room: "Arts Building · 18",
0085        },
0086        {
0087          id: "c5",
0088          day: "Tue",
0089          time: "10:00",
0090          name: "Biology 101",
0091          room: "Science Hall · 204",
0092        },
0093        {
0094          id: "c6",
0095          day: "Tue",
0096          time: "1:00",
0097          name: "Physical Education",
0098          room: "Gymnasium",
0099        },
0100        {
0101          id: "c7",
0102          day: "Wed",
0103          time: "9:00",
0104          name: "Biology 101",
0105          room: "Science Hall · 204",
0106        },
0107        {
0108          id: "c8",
0109          day: "Wed",
0110          time: "11:00",
0111          name: "Literature & Society",
0112          room: "Arts Building · 12",
0113        },
0114        {
0115          id: "c9",
0116          day: "Wed",
0117          time: "2:00",
0118          name: "Statistics",
0119          room: "Main Building · 305",
0120        },
0121        {
0122          id: "c10",
0123          day: "Thu",
0124          time: "8:00",
```

app.js - lines 125-186

```
0125          name: "Communication",
0126          room: "Arts Building · 18",
0127        },
0128        {
0129          id: "c11",
0130          day: "Thu",
0131          time: "10:00",
0132          name: "Biology 101",
0133          room: "Science Hall · 204",
0134        },
0135        {
0136          id: "c12",
0137          day: "Fri",
0138          time: "9:00",
0139          name: "Literature & Society",
0140          room: "Arts Building · 12",
0141        },
0142        {
0143          id: "c13",
0144          day: "Fri",
0145          time: "11:00",
0146          name: "Statistics",
0147          room: "Main Building · 305",
0148        },
0149      ],
0150      expenses: [
0151        {
0152          id: "e1",
0153          name: "Lunch at campus",
0154          category: "Food",
0155          amount: 145,
0156          date: "2026-10-17",
0157        },
0158        {
0159          id: "e2",
0160          name: "Jeepney fare",
0161          category: "Transport",
0162          amount: 40,
0163          date: "2026-10-16",
0164        },
0165        {
0166          id: "e3",
0167          name: "Notebook & pens",
0168          category: "School",
0169          amount: 185,
0170          date: "2026-10-15",
0171        },
0172        {
0173          id: "e4",
0174          name: "Iced coffee",
0175          category: "Food",
0176          amount: 120,
0177          date: "2026-10-14",
0178        },
0179      ],
0180      gradesComputed: false,
0181      grades: [
0182        { id: "g1", category: "Quiz", score: 18, total: 20 },
0183        { id: "g2", category: "Quiz", score: 15, total: 20 },
0184        { id: "g3", category: "Long Quiz", score: 42, total: 50 },
0185        { id: "g4", category: "Assignment", score: 27, total: 30 },
0186        { id: "g5", category: "Exam", score: 84, total: 100 },
```

app.js - lines 187-248

```
0187        { id: "g6", category: "Performance Task", score: 28, total: 30 },
0188      ],
0189    };
0190
0191    // 2. Saved state and small helpers
0192    // Short DOM helpers keep the render functions readable.
0193    const $ = (s, root = document) => root.querySelector(s);
0194    const $$ = (s, root = document) => [...root.querySelectorAll(s)];
0195    // Convert older 12-hour text values and new time-picker values to HH:mm.
0196    function normalizeClassTime(value) {
0197      const match = String(value || "").trim().match(/^(\d{1,2}):(\d{2})(?:\s*(AM|PM))?$/i);
0198      if (!match) return "09:00";
0199      let hour = Number(match[1]);
0200      const minute = Number(match[2]);
0201      const period = match[3]?.toUpperCase();
0202      if (period === "PM" && hour < 12) hour += 12;
0203      if (period === "AM" && hour === 12) hour = 0;
0204      // Earlier sample data used 1:00-4:00 to mean 1:00-4:00 PM.
0205      if (!period && match[1].length === 1 && hour >= 1 && hour <= 4) hour += 12;
0206      if (hour > 23 || minute > 59) return "09:00";
0207      return `${String(hour).padStart(2, "0")}:${String(minute).padStart(2, "0")}`;
0208    }
0209    let state;
0210    try {
0211      const saved = JSON.parse(localStorage.getItem(KEY));
0212      state = saved ? { ...structuredClone(seed), ...saved } : structuredClone(seed);
0213      state.gradeWeights = { ...seed.gradeWeights, ...(state.gradeWeights || {}) };
0214      state.classes = (Array.isArray(state.classes) ? state.classes : structuredClone(seed.classes))
0215        .map((item) => ({ ...item, time: normalizeClassTime(item.time) }));
0216      if (!Array.isArray(state.grades)) state.grades = structuredClone(seed.grades);
0217      // Keep old subject grades in storage before switching to assessment rows.
0218      if (state.grades?.some((grade) => !("category" in grade))) {
0219        state.previousSubjectGrades = structuredClone(state.grades);
0220        state.grades = structuredClone(seed.grades);
0221        state.gradesComputed = false;
0222      }
0223      if (typeof state.gradesComputed !== "boolean") state.gradesComputed = false;
0224    } catch {
0225      state = structuredClone(seed);
0226    }
0227    // Save the complete planner snapshot after each change so the sections stay in sync.
0228    const persist = () => localStorage.setItem(KEY, JSON.stringify(state));
0229    const money = (value) =>
0230      new Intl.NumberFormat("en-PH", {
0231        style: "currency",
0232        currency: "PHP",
0233        maximumFractionDigits: 0,
0234      }).format(Number(value) || 0);
0235    const EXPENSE_ICONS = {
0236      Food: " ",
0237      Transport: " ",
0238      School: " ",
0239      Personal: " ",
0240      Other: "·",
0241    };
0242    const escapeHtml = (value) =>
0243      String(value).replace(
0244        /[&<>"']/g,
0245        (c) =>
0246          ({
0247            "&": "&amp;",
0248            "<": "&lt;",
```

app.js - lines 249-302

```
0249            ">": "&gt;",
0250            '"': "&quot;",
0251            "'": "&#39;",
0252          })[c],
0253      );
0254    const makeId = () =>
0255      `${Date.now().toString(36)}${Math.random().toString(36).slice(2, 7)}`;
0256    const today = new Date();
0257    const friendlyDate = today.toLocaleDateString("en-PH", {
0258      weekday: "long",
0259      month: "long",
0260      day: "numeric",
0261    });
0262    $("#today-label").textContent = friendlyDate;
0263    $("#planner-date").textContent = `TODAY · ${friendlyDate.toUpperCase()}`;
0264    $("#greeting").textContent =
0265      `${today.getHours() < 12 ? "GOOD MORNING" : today.getHours() < 18 ? "GOOD AFTERNOON" : "GOOD
      EVENING"}, JAMIE`;
0266    const toast = (message) => {
0267      const node = $("#toast");
0268      node.textContent = message;
0269      node.classList.add("show");
0270      clearTimeout(toast.timer);
0271      toast.timer = setTimeout(() => node.classList.remove("show"), 2200);
0272    };
0273
0274    // 3. Rendering helpers
0275    // These functions turn saved state into the cards, tables, and totals on screen.
0276    // Shared markup keeps task rows consistent on the dashboard and full planner page.
0277    function tasksMarkup(tasks, controls = true) {
0278      if (!tasks.length)
0279        return '<div class="empty-state">Nothing here just yet. Add a task when you’re ready.</div>';
0280      return tasks
0281        .map(
0282          (task) =>
0283            `<div class="task-row ${task.done ? "task-done" : ""}"><button class="check-button
      ${task.done ? "checked" : ""}" data-toggle-task="${task.id}" aria-label="${task.done ? "Mark
      incomplete" : "Mark complete"}">${task.done ? " " : ""}</button><div
      class="task-copy"><strong>${escapeHtml(task.title)}</strong><small>${escapeHtml(task.due ||
      "Today")}</small></div><span class="priority ${(task.priority ||
      "Medium").toLowerCase()}">${escapeHtml(task.priority || "Medium")}</span>${controls ? `<span
      class="task-actions"><button class="tiny-action" data-delete-task="${task.id}"
      aria-label="Delete task">×</button></span>` : ""}</div>`,
0284        )
0285        .join("");
0286    }
0287    function getRowGrade(row) {
0288      if (row.score === "" || row.score == null || row.total === "" || row.total == null)
0289        return null;
0290      const score = Number(row.score);
0291      const total = Number(row.total);
0292      return Number.isFinite(score) && Number.isFinite(total) && total > 0
0293        ? (score / total) * 100
0294        : null;
0295    }
0296    function getGradeWeightTotal() {
0297      return GRADE_CATEGORIES.reduce(
0298        (sum, category) => sum + (Number(state.gradeWeights[category]) || 0),
0299        0,
0300      );
0301    }
0302    function getGradedWeight() {
```

app.js - lines 303-364

```
0303      return GRADE_CATEGORIES.reduce((sum, category) => {
0304        const hasScores = state.grades.some(
0305          (row) => row.category === category && getRowGrade(row) !== null,
0306        );
0307        return sum + (hasScores ? Number(state.gradeWeights[category]) || 0 : 0);
0308      }, 0);
0309    }
0310    // Weight each category's points average by the percentage from the syllabus.
0311    function getOverallRawGrade() {
0312      if (Math.abs(getGradeWeightTotal() - 100) > 0.01) return null;
0313      let weightedTotal = 0;
0314      let includedWeight = 0;
0315      for (const category of GRADE_CATEGORIES) {
0316        const weight = Number(state.gradeWeights[category]) || 0;
0317        const rows = state.grades.filter(
0318          (row) => row.category === category && getRowGrade(row) !== null,
0319        );
0320        if (!rows.length || weight <= 0) continue;
0321        const earned = rows.reduce((sum, row) => sum + Number(row.score), 0);
0322        const possible = rows.reduce((sum, row) => sum + Number(row.total), 0);
0323        if (possible <= 0) continue;
0324        weightedTotal += (earned / possible) * 100 * weight;
0325        includedWeight += weight;
0326      }
0327      return includedWeight > 0 ? weightedTotal / includedWeight : null;
0328    }
0329    function renderTasks() {
0330      const remaining = state.tasks.filter((t) => !t.done).length,
0331        done = state.tasks.length - remaining;
0332      $("#stat-tasks").innerHTML =
0333        `${remaining} <small>${remaining === 1 ? "task" : "tasks"} left</small>`;
0334      $("#today-count").textContent = state.tasks.length;
0335      $("#planner-count").textContent = remaining;
0336      $("#task-foot").textContent = state.tasks.length
0337        ? `${done} of ${state.tasks.length} completed`
0338        : "Add a task to get started";
0339      $("#task-progress").style.width =
0340        `${state.tasks.length ? (done / state.tasks.length) * 100 : 0}%`;
0341      $("#overview-tasks").innerHTML = tasksMarkup(state.tasks.slice(0, 4));
0342      const filter = $(".filter-tab.selected")?.dataset.filter || "all";
0343      $("#planner-tasks").innerHTML = tasksMarkup(
0344        state.tasks.filter(
0345          (t) => filter === "all" || (filter === "open" ? !t.done : t.done),
0346        ),
0347      );
0348    }
0349    function renderBudget() {
0350      const allowance = Number(state.allowance) || 0;
0351      const spent = state.expenses.reduce((sum, e) => sum + Number(e.amount), 0),
0352        remaining = allowance - spent,
0353        ratio =
0354          allowance > 0
0355            ? Math.min(100, (spent / allowance) * 100)
0356            : 100;
0357      $("#stat-budget").textContent = money(remaining);
0358      $("#budget-foot").textContent =
0359        `of your ${money(allowance)} monthly allowance`;
0360      $("#overview-spent").textContent = money(spent);
0361      $("#spent-label").textContent = `${money(spent)} spent`;
0362      $("#limit-label").textContent = `${money(allowance)} limit`;
0363      $("#budget-bar").style.width = `${ratio}%`;
0364      $("#budget-total").textContent = money(allowance);
```

app.js - lines 365-413

```
0365      $("#budget-spent").textContent = money(spent);
0366      $("#budget-remaining").textContent = money(remaining);
0367      $("#budget-remaining").style.color = remaining < 0 ? "#ce8065" : "";
0368      $("#budget-remaining-note").textContent =
0369        remaining < 0 ? "Over your monthly allowance" : "You’re doing great";
0370      $("#rule-allowance").textContent = money(allowance);
0371      $("#rule-needs").textContent = money(allowance * 0.5);
0372      $("#rule-wants").textContent = money(allowance * 0.3);
0373      $("#rule-savings").textContent = money(allowance * 0.2);
0374      const icons = EXPENSE_ICONS;
0375      $("#recent-transactions").innerHTML =
0376        state.expenses
0377          .slice(0, 2)
0378          .map(
0379            (e) =>
0380              `<div class="transaction-row"><span class="transaction-icon">${icons[e.category] ||
      "•"}</span><span class="transaction-info"><strong>${escapeHtml(e.name)}</strong><small>${escap
      eHtml(e.category)}</small></span><span
      class="transaction-amount">−${money(e.amount)}</span></div>`,
0381          )
0382          .join("") || '<div class="empty-state">No expenses yet.</div>';
0383      renderExpenses();
0384    }
0385    function renderExpenses() {
0386      const filter = $("#expense-filter").value;
0387      const rows = state.expenses.filter(
0388        (e) => filter === "all" || e.category === filter,
0389      );
0390      const icons = EXPENSE_ICONS;
0391      $("#expense-list").innerHTML = rows.length
0392        ? rows
0393            .map(
0394              (e) =>
0395                `<div class="expense-row"><span class="expense-icon">${icons[e.category] ||
      "•"}</span><span class="expense-copy"><strong>${escapeHtml(e.name)}</strong><small>${new
      Date(`${e.date}T12:00:00`).toLocaleDateString("en-PH", { month: "short", day: "numeric", year:
       "numeric" })}</small></span><span
      class="expense-category">${escapeHtml(e.category)}</span><span
      class="expense-amount">−${money(e.amount)}</span></div>`,
0396            )
0397            .join("")
0398        : '<div class="empty-state">No expenses in this category.</div>';
0399    }
0400    function formatPercent(value) {
0401      return `${Number(Number(value).toFixed(1))}%`;
0402    }
0403    function renderGradeWeights() {
0404      $("#grade-weights").innerHTML = GRADE_CATEGORIES.map((category, index) => {
0405        const inputId = `grade-weight-${index}`;
0406        return `<div class="grade-weight-card"><label
      for="${inputId}">${escapeHtml(category)}</label><div class="grade-weight-input-wrap"><input
      id="${inputId}" type="number" min="0" max="100" step="0.1"
      value="${escapeHtml(state.gradeWeights[category] ?? 0)}"
      data-grade-weight="${escapeHtml(category)}" aria-label="${escapeHtml(category)} weight
      percentage"><span>%</span></div></div>`;
0407      }).join("");
0408    }
0409    function renderGradeSummary() {
0410      const weightTotal = getGradeWeightTotal();
0411      const weightsValid = Math.abs(weightTotal - 100) <= 0.01;
0412      const gradedWeight = getGradedWeight();
0413      const avg = state.gradesComputed && weightsValid ? getOverallRawGrade() : null;
```

app.js - lines 414-462

```
0414      $("#grade-weight-total").textContent = formatPercent(weightTotal);
0415      $("#grade-weight-status").textContent = weightsValid
0416        ? gradedWeight > 0
0417          ? `Weights total 100%. Scores currently cover ${formatPercent(gradedWeight)} of the course
      weight.`
0418          : "Weights total 100%. Add scores in one or more weighted categories."
0419        : `Weights total ${formatPercent(weightTotal)}. Adjust them to 100% before calculating.`;
0420      $("#grade-weight-status").classList.toggle("is-valid", weightsValid);
0421      $("#stat-grade").innerHTML =
0422        avg === null
0423          ? "— <small>%</small>"
0424          : `${avg.toFixed(1)}<small>%</small>`;
0425      $("#grade-average").innerHTML =
0426        avg === null ? "—<small>%</small>" : `${avg.toFixed(1)}<small>%</small>`;
0427      const resultMessage = avg >= 90
0428        ? "Excellent work — your effort is showing."
0429        : avg >= 75
0430          ? "You’re making steady progress. Keep going."
0431          : "Use this result to guide your next study session.";
0432      $("#grade-caption").textContent = avg === null
0433        ? weightsValid
0434          ? "Enter scores and totals, then compute your weighted grade."
0435          : "Set category weights that add up to 100% before calculating."
0436        : `Based on scores covering ${formatPercent(gradedWeight)} of your course weight.
      ${resultMessage}`;
0437      $("#grade-count").textContent =
0438        `${state.grades.length} ${state.grades.length === 1 ? "row" : "rows"}`;
0439    }
0440    function renderGrades() {
0441      renderGradeWeights();
0442      renderGradeSummary();
0443      $("#grade-rows").innerHTML = state.grades.length
0444        ? state.grades
0445            .map(
0446              (row) => {
0447                const rawGrade = getRowGrade(row);
0448                return `<tr><td><select data-grade-row="${row.id}" data-grade-field="category"
      aria-label="Assessment category">${GRADE_CATEGORIES.map((category) => `<option
      value="${category}" ${row.category === category ? "selected" :
      ""}>${category}</option>`).join("")}</select></td><td><input type="number" min="0" step="any"
      value="${row.score ?? ""}" placeholder="Score" data-grade-row="${row.id}"
      data-grade-field="score" aria-label="Score"></td><td><input type="number" min="0.01"
      step="any" value="${row.total ?? ""}" placeholder="Total" data-grade-row="${row.id}"
      data-grade-field="total" aria-label="Total possible points"></td><td
      class="grade-raw">${state.gradesComputed && rawGrade !== null ? `${rawGrade.toFixed(1)}%` :
      "—"}</td><td><button class="row-action" data-delete-grade="${row.id}" aria-label="Remove grade
       row">×</button></td></tr>`;
0449              },
0450            )
0451            .join("")
0452        : '<tr><td colspan="5"><div class="empty-state">No grade rows yet. Add a row to get
      started.</div></td></tr>';
0453    }
0454    // Compare class times as minutes after midnight so custom times sort correctly.
0455    function toMinutes(time) {
0456      const [hour, minute] = normalizeClassTime(time).split(":").map(Number);
0457      return hour * 60 + minute;
0458    }
0459    function formatTime(time) {
0460      const [hour, minute] = normalizeClassTime(time).split(":").map(Number);
0461      const displayHour = hour % 12 || 12;
0462      return `${displayHour}:${String(minute).padStart(2, "0")} ${hour < 12 ? "AM" : "PM"}`;
```

app.js - lines 463-515

```
  }
  function renderClasses() {
    const grid = $("#timetable");
    grid.innerHTML = `<div class="time-head">TIME</div>${DAYS.map((d) => `<div class="day-head">${d.toUpperCase()}</div>`).join("")}`;
    const classTimes = [...new Set([
      ...HOURS,
      ...state.classes.map((item) => normalizeClassTime(item.time)),
    ])].sort((a, b) => toMinutes(a) - toMinutes(b));
    for (const time of classTimes) {
      grid.insertAdjacentHTML(
        "beforeend",
        `<div class="time-slot">${formatTime(time)}</div>`,
      );
      for (const day of DAYS) {
        const items = state.classes.filter(
          (item) => item.day === day && normalizeClassTime(item.time) === time,
        );
        const blocks = items.map((item) =>
          `<div class="class-block" title="Click to remove this class" data-delete-class="${item.id}"><strong>${escapeHtml(item.name)}</strong><small>${escapeHtml(formatTime(item.time))} · ${escapeHtml(item.room)}</small></div>`,
        ).join("");
        grid.insertAdjacentHTML(
          "beforeend",
          `<div class="class-slot">${blocks}</div>`,
        );
      }
    }
    const todayCode = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"][
      today.getDay()
    ];
    const now = today.getHours() * 60 + today.getMinutes();
    const upcoming = state.classes
      .filter((c) => c.day === todayCode && toMinutes(c.time) >= now)
      .sort((a, b) => toMinutes(a.time) - toMinutes(b.time))[0];
    const next = $("#next-class");
    next.innerHTML = upcoming
      ? `<div class="class-card"><span class="class-time">${escapeHtml(formatTime(upcoming.time))}</span><span class="class-stripe"></span><span class="class-info"><strong>${escapeHtml(upcoming.name)}</strong><small>${escapeHtml(upcoming.room)}</small></span></div><div class="no-class">Your next class today.</div>`
      : `<div class="no-class">${DAYS.includes(todayCode) ? "No more classes today. Take a breather." : "No classes scheduled today. Enjoy your day."}</div>`;
  }
  function renderWeek() {
    const start = new Date(today);
    start.setDate(today.getDate() - ((today.getDay() + 6) % 7));
    $("#week-mini").innerHTML = Array.from({ length: 7 }, (_, i) => {
      const date = new Date(start);
      date.setDate(start.getDate() + i);
      return `<div class="mini-day ${i === (today.getDay() + 6) % 7 ? "current" : ""}">${["M", "T", "W", "T", "F", "S", "S"][i]}<b>${date.getDate()}</b></div>`;
    }).join("");
  }
  function render() {
    renderTasks();
    renderBudget();
    renderGrades();
    renderClasses();
    renderWeek();
```

app.js - lines 516-577

```
0516      persist();
0517    }
0518
0519    // 4. Navigation and page interactions
0520    function navigate(page) {
0521      if (!$("#page-" + page)) page = "overview";
0522      $$(".page-section").forEach((n) =>
0523        n.classList.toggle("active-page", n.id === `page-${page}`),
0524      );
0525      $$(".nav-link").forEach((n) =>
0526        n.classList.toggle("active", n.dataset.page === page),
0527      );
0528      $("#crumb-current").textContent = {
0529        overview: "Overview",
0530        planner: "Daily planner",
0531        classes: "Class schedule",
0532        budget: "My budget",
0533        grades: "Grade calculator",
0534      }[page];
0535      history.replaceState(null, "", `#${page}`);
0536      $("#sidebar").classList.remove("open");
0537    }
0538    $$(".nav-link").forEach((link) =>
0539      link.addEventListener("click", (e) => {
0540        e.preventDefault();
0541        navigate(link.dataset.page);
0542      }),
0543    );
0544    $("#menu-toggle").addEventListener("click", () =>
0545      $("#sidebar").classList.toggle("open"),
0546    );
0547    // Delegate clicks because task, class, and grade controls are rebuilt during rendering.
0548    document.addEventListener("click", (e) => {
0549      const target = e.target.closest("[data-action]");
0550      if (target) openDialog(target.dataset.action);
0551      const toggle = e.target.closest("[data-toggle-task]");
0552      if (toggle) {
0553        const task = state.tasks.find((t) => t.id === toggle.dataset.toggleTask);
0554        if (task) task.done = !task.done;
0555        render();
0556      }
0557      const del = e.target.closest("[data-delete-task]");
0558      if (del) {
0559        state.tasks = state.tasks.filter((t) => t.id !== del.dataset.deleteTask);
0560        render();
0561        toast("Task removed");
0562      }
0563      const gradeDel = e.target.closest("[data-delete-grade]");
0564      if (gradeDel) {
0565        state.grades = state.grades.filter(
0566          (g) => g.id !== gradeDel.dataset.deleteGrade,
0567        );
0568        state.gradesComputed = false;
0569        render();
0570        toast("Grade row removed");
0571      }
0572      const classDel = e.target.closest("[data-delete-class]");
0573      if (classDel) {
0574        state.classes = state.classes.filter(
0575          (c) => c.id !== classDel.dataset.deleteClass,
0576        );
0577        render();
```

app.js - lines 578-639

```
0578        toast("Class removed");
0579      }
0580    });
0581    $$(".filter-tab").forEach((tab) =>
0582      tab.addEventListener("click", () => {
0583        $$(".filter-tab").forEach((t) =>
0584          t.classList.toggle("selected", t === tab),
0585        );
0586        renderTasks();
0587      }),
0588    );
0589    $("#expense-filter").addEventListener("change", renderExpenses);
0590    $("#grade-weights").addEventListener("change", (e) => {
0591      const input = e.target.closest("[data-grade-weight]");
0592      if (!input) return;
0593      const category = input.dataset.gradeWeight;
0594      state.gradeWeights[category] = input.value === ""
0595        ? ""
0596        : Math.min(100, Math.max(0, Number(input.value) || 0));
0597      state.gradesComputed = false;
0598      renderGradeSummary();
0599      persist();
0600    });
0601    $("#grade-rows").addEventListener("change", (e) => {
0602      const field = e.target.closest("[data-grade-row]");
0603      if (!field) return;
0604      const row = state.grades.find((g) => g.id === field.dataset.gradeRow);
0605      if (row) {
0606        const property = field.dataset.gradeField;
0607        row[property] = property === "category"
0608          ? field.value
0609          : field.value === ""
0610            ? ""
0611            : Math.max(0, Number(field.value) || 0);
0612        state.gradesComputed = false;
0613        if (property !== "category") {
0614          $$(".grade-raw").forEach((cell) => (cell.textContent = "—"));
0615        }
0616        renderGradeSummary();
0617        persist();
0618      }
0619    });
0620    $("#add-grade-row").addEventListener("click", () => {
0621      state.grades.push({
0622        id: makeId(),
0623        category: GRADE_CATEGORIES[0],
0624        score: "",
0625        total: "",
0626      });
0627      state.gradesComputed = false;
0628      renderGrades();
0629      persist();
0630      $('#grade-rows tr:last-child [data-grade-field="score"]')?.focus();
0631    });
0632    $("#compute-grade").addEventListener("click", () => {
0633      if (Math.abs(getGradeWeightTotal() - 100) > 0.01) {
0634        state.gradesComputed = false;
0635        renderGradeSummary();
0636        toast("Adjust category weights until they total 100%");
0637        return;
0638      }
0639      if (getGradedWeight() <= 0) {
```

app.js - lines 640-689

```
0640        toast("Enter a score in a category with a weight above 0%");
0641        return;
0642      }
0643      state.gradesComputed = true;
0644      renderGrades();
0645      persist();
0646    });
0647
0648    // 5. Shared entry dialog
0649    // Cancel and close only dismiss the dialog; the form submit handler is reserved for Save.
0650    const dialog = $("#entry-dialog"),
0651      form = $("#entry-form");
0652    document.addEventListener("click", (e) => {
0653      if (e.target.closest("[data-close-dialog]")) dialog.close();
0654    });
0655    function field(
0656      label,
0657      name,
0658      type = "text",
0659      value = "",
0660      options = null,
0661      extra = "",
0662    ) {
0663      if (options)
0664        return `<div class="dialog-field"><label for="field-${name}">${label}</label><select
      id="field-${name}" name="${name}" ${extra}>${options.map((o) => `<option
      value="${escapeHtml(o)}" ${o === value ? "selected" :
      ""}>${escapeHtml(o)}</option>`).join("")}</select></div>`;
0665      return `<div class="dialog-field"><label for="field-${name}">${label}</label><input
      id="field-${name}" name="${name}" type="${type}" value="${escapeHtml(value)}" ${type ===
      "number" ? 'min="0" step="1"' : ""} ${extra} required></div>`;
0666    }
0667    // One dialog builder supplies the fields and save action for each add/edit interaction.
0668    function openDialog(action) {
0669      const config = {
0670        "add-task": {
0671          title: "Add a task",
0672          subtitle: "A SMALL THING, DONE WELL",
0673          fields: `${field("What needs doing?", "title", "text", "", "", 'placeholder="e.g. Review
      lecture notes"')}<div class="dialog-grid">${field("Due", "due", "text", "Today · Anytime", "",
       'placeholder="Today · 3:00 PM"')}${field("Priority", "priority", "text", "Medium", ["Low",
      "Medium", "High"])}</div>`,
0674          save: (data) =>
0675            state.tasks.unshift({
0676              id: makeId(),
0677              title: data.get("title").trim(),
0678              due: data.get("due").trim() || "Today",
0679              priority: data.get("priority"),
0680              done: false,
0681            }),
0682        },
0683        "add-expense": {
0684          title: "Add an expense",
0685          subtitle: "MONEY CHECK-IN",
0686          fields: `${field("What did you spend on?", "name", "text", "", "", 'placeholder="e.g.
      Lunch"')}<div class="dialog-grid">${field("Amount (₱)", "amount", "number", "", "",
      'min="1"')}${field("Category", "category", "text", "Food", ["Food", "Transport", "School",
      "Personal", "Other"])}</div>${field("Date", "date", "date", today.toISOString().slice(0,
      10))}`,
0687          save: (data) =>
0688            state.expenses.unshift({
0689              id: makeId(),
```

app.js - lines 690-748

```
0690              name: data.get("name").trim(),
0691              amount: Number(data.get("amount")),
0692              category: data.get("category"),
0693              date: data.get("date"),
0694            }),
0695        },
0696        "add-class": {
0697          title: "Add a class",
0698          subtitle: "BUILD YOUR WEEK",
0699          fields: `${field("Subject", "name", "text", "", "", 'placeholder="e.g. Chemistry"')}<div
      class="dialog-grid">${field("Day", "day", "text", "Mon", DAYS)}${field("Start time", "time",
      "time", "09:00")}</div>${field("Room or location", "room", "text", "", "", 'placeholder="e.g.
      Science Hall · 204"')}`,
0700          save: (data) =>
0701            state.classes.push({
0702              id: makeId(),
0703              day: data.get("day"),
0704              time: normalizeClassTime(data.get("time")),
0705              name: data.get("name").trim(),
0706              room: data.get("room").trim() || "Room not set",
0707            }),
0708        },
0709        "edit-budget": {
0710          title: "Monthly allowance",
0711          subtitle: "SET YOUR MONTHLY PLAN",
0712          fields: field(
0713            "Allowance (₱)",
0714            "allowance",
0715            "number",
0716            state.allowance,
0717            "",
0718            'min="0"',
0719          ),
0720          save: (data) => (state.allowance = Number(data.get("allowance"))),
0721        },
0722      }[action];
0723      if (!config) return;
0724      $("#dialog-title").textContent = config.title;
0725      $("#dialog-eyebrow").textContent = config.subtitle;
0726      $("#dialog-fields").innerHTML = config.fields;
0727      form.onsubmit = (e) => {
0728        e.preventDefault();
0729        if (!form.reportValidity()) return;
0730        const data = new FormData(form);
0731        config.save(data);
0732        render();
0733        dialog.close();
0734        toast(
0735          action === "edit-budget"
0736            ? "Allowance updated"
0737            : "Added to your planner",
0738        );
0739      };
0740      dialog.showModal();
0741      setTimeout(() => $("#dialog-fields input")?.focus(), 30);
0742    }
0743
0744    // 6. Reset and first page load
0745    $("#reset-data").addEventListener("click", () => {
0746      if (
0747        confirm(
0748          "Restore the original sample planner? Your current changes will be replaced.",
```

app.js - lines 749-764

```
0749        )
0750      ) {
0751        state = structuredClone(seed);
0752        render();
0753        navigate("overview");
0754        toast("Sample planner restored");
0755      }
0756    });
0757    const initial = location.hash.replace("#", "");
0758    navigate(
0759      ["overview", "planner", "classes", "budget", "grades"].includes(initial)
0760        ? initial
0761        : "overview",
0762    );
0763    render();
0764  })();
```